Training Models

Book:

“Hands-on Machine Learning with Scikit –Learn, Keras and Tensorflow”, Aurelion Geron

Chapter 4: Training Models



Outline

- Linear Regression
  - Normal equation
  - Complexity
- Gradient Descent
  - Batch GD,
  - Stochastic GD,
  - Mini-batch GD

- Polynomial Regression
- Learning Curves
- Regularized Linear Models
  - Ridge regression
  - Lasso regression
  - Elastic Net
  - Early Stopping

- Logistic Regression
  - Estimating probabilities
  - Training and cost function
  - Decision boundaries
  - Softmax Regression

Create the Workspace - Python

- Installation and tutorials of Python:
  - https://www.python.org/
- Python Modules: Jupyter, NumPy, pandas, Matplotlib, Scikit-Learn
- For creating an isolated environment: see the book (2nd edition) pages 75-79
- All the code used in the book is available from:
  - https://github.com/ageron/handson-ml2
- NEW: Third edition on the book is also available in 2023.
  - https://github.com/ageron/handson-ml3

Notations

- m is the number of instances in the dataset
- $\mathbf{x}^{(i)}$ is a vector of all the feature values (excluding the label) of the ith instance in the dataset, and $y^{(i)}$ is its label (the desired output value for that instance).
- $\mathbf{X}$ is a matrix containing all the feature values (excluding labels) of all instances in the dataset. There is one row per instance, and the ith row is equal to the transpose of $\mathbf{x}^{(i)}$, noted $(\mathbf{x}^{(i)})^{\intercal}$.[4]
- h is your system's prediction function, also called a *hypothesis*. When your system is given an instance's feature vector $\mathbf{x}^{(i)}$, it outputs a predicted value $\hat{y}^{(i)} = h(\mathbf{x}^{(i)})$ for that instance ($\hat{y}$ is pronounced "y-hat").

$$\mathbf{X} = \begin{pmatrix} (\mathbf{x}^{(1)})^{\intercal} \\ (\mathbf{x}^{(2)})^{\intercal} \\ \vdots \\ (\mathbf{x}^{(1999)})^{\intercal} \\ (\mathbf{x}^{(2000)})^{\intercal} \end{pmatrix}$$

Linear Regression

Normal Equation

Linear Regression

- A linear model makes a prediction by simply computing a weighted sum of the input features, plus a constant called the *bias term* (also called the *intercept term*) $\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$
  - $\hat{y}$ is the predicted value.
  - n is the number of features.
  - x_i is the ith feature value.
  - θ_j is the jth model parameter (including the bias term θ_0 and the feature weights $\theta_1, \theta_2, \cdots, \theta_n$).

Linear Regression

- Vectorized form: $$\hat{y} = h_{\boldsymbol{\theta}}(\mathbf{x}) = \boldsymbol{\theta} \cdot \mathbf{x}$$
  - $\boldsymbol{\theta}$ is the model's *parameter vector*, containing the bias term θ_0 and the feature weights θ_1 to θ_n.
  - $\mathbf{x}$ is the instance's *feature vector*, containing x_0 to x_n, with x_0 always equal to 1.
  - $\boldsymbol{\theta} \cdot \mathbf{x}$ is the dot product of the vectors $\boldsymbol{\theta}$ and $\mathbf{x}$, which is of course equal to $\theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$.
  - $h_{\boldsymbol{\theta}}$ is the hypothesis function, using the model parameters $\boldsymbol{\theta}$.

Linear Regression – Cost Function

The MSE of a Linear Regression hypothesis $h_{\boldsymbol{\theta}}$ on a training set $\mathbf{X}$ is calculated using Equation

$$\operatorname{MSE}(\mathbf{X}, h_{\boldsymbol{\theta}}) = \frac{1}{m} \sum_{i=1}^{m} \left(\boldsymbol{\theta}^{\mathsf{T}} \mathbf{x}^{(i)} - y^{(i)} \right)^2$$

write MSE($\boldsymbol{\theta}$) instead of MSE($\mathbf{X}$, $h_{\boldsymbol{\theta}}$)

Linear Regression – Training

- To train a Linear Regression model, we need to find the value of $\boldsymbol{\theta}$ that minimizes the MSE.

Linear Regression – The Normal Equation

- The normal equation in a closed-form solution that gives the result directly.

$$\widehat{\boldsymbol{\theta}} = (\mathbf{X}^{\intercal}\mathbf{X})^{-1}\,\mathbf{X}^{\intercal}\,\mathbf{y}$$

In this equation:

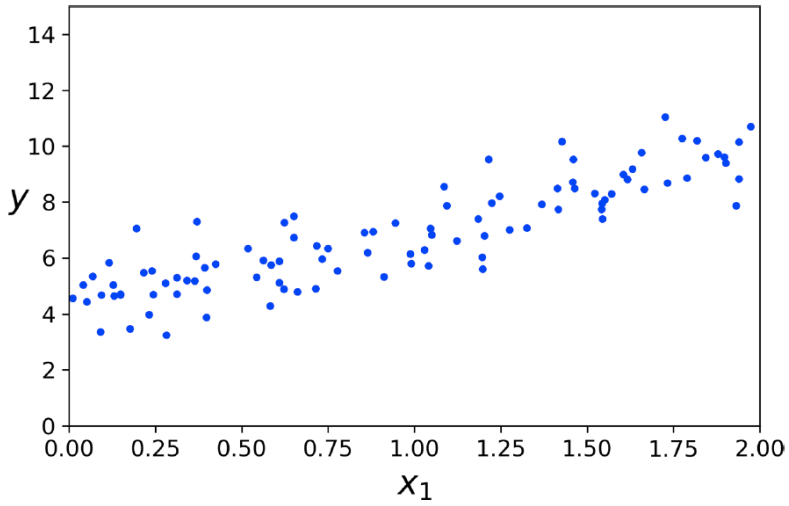
- $\widehat{\boldsymbol{\theta}}$ is the value of $\boldsymbol{\theta}$ that minimizes the cost function.
- $\mathbf{y}$ is the vector of target values containing $y^{(1)}$ to $y^{(m)}$.

Linear Regression – The Normal Equation
Implementation

Let's generate some linear-looking data to test this equation on (Figure

```
import numpy as np

np.random.seed(42)  # to make this code example reproducible
m = 100  # number of instances
X = 2 * np.random.rand(m, 1)  # column vector
y = 4 + 3 * X + np.random.randn(m, 1)  # column vector
```



Linear Regression – The Normal Equation

Implementation using Numpy

$$\widehat{\boldsymbol{\theta}} = (\mathbf{X}^\intercal \mathbf{X})^{-1}\, \mathbf{X}^\intercal\, \mathbf{y}$$

Now let's compute $\widehat{\boldsymbol{\theta}}$ using the Normal Equation. We will use the `inv()` function from NumPy's linear algebra module (`np.linalg`) to compute the inverse of a matrix, and the `dot()` method for matrix multiplication:

column-wise concatenation

```
X_b = np.c_[np.ones((100, 1)), X]  # add x0 = 1 to each instance
theta_best = np.linalg.inv(X_b.T.dot(X_b)).dot(X_b.T).dot(y)
```

OR

```
theta_best = np.linalg.inv(X_b.T @ X_b) @ X_b.T @ y
```

The `@` operator performs matrix multiplication. If `A` and `B` are NumPy arrays, then `A @ B` is equivalent to `np.matmul(A, B)`.

Linear Regression – The Normal Equation

Implementation using Numpy

The function that we used to generate the data is $y = 4 + 3x_1 + \text{Gaussian noise}$. Let's see what the equation found:

```
>>> theta_best
array([[4.21509616],
       [2.77011339]])
```

We would have hoped for $\theta_0 = 4$ and $\theta_1 = 3$ instead of $\theta_0 = 4.215$ and $\theta_1 = 2.770$. Close enough, but the noise made it impossible to recover the exact parameters of the original function.

Linear Regression – The Normal Equation

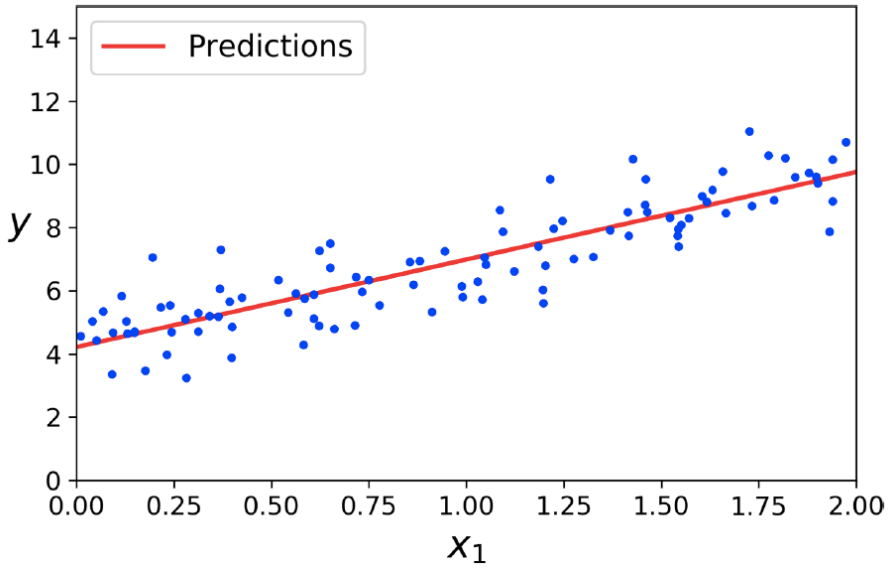
Implementation using Numpy

Now we can make predictions using $\widehat{\boldsymbol{\theta}}$:

```
>>> X_new = np.array([[0], [2]])
>>> X_new_b = np.c_[np.ones((2, 1)), X_new] # add x0 = 1 to each instance
>>> y_predict = X_new_b.dot(theta_best)
>>> y_predict
array([[4.21509616],
       [9.75532293]])
```

Let's plot this model's predictions (Figure 4-2):

```
plt.plot(X_new, y_predict, "r-")
plt.plot(X, y, "b.")
plt.axis([0, 2, 0, 15])
plt.show()
```



Linear Regression – The Normal Equation

Implementation using Scikit-Learn

- Implement using Scikit-Learn

```
>>> from sklearn.linear_model import LinearRegression
>>> lin_reg = LinearRegression()
>>> lin_reg.fit(X, y)
>>> lin_reg.intercept_, lin_reg.coef_
(array([4.21509616]), array([[2.77011339]]))
>>> lin_reg.predict(X_new)
array([[4.21509616],
       [9.75532293]])
```

Linear Regression – The Normal Equation

Implementation using Scikit-Learn

The `LinearRegression` class is based on the `scipy.linalg.lstsq()` function (the name stands for "least squares"), which you could call directly:

```
>>> theta_best_svd, residuals, rank, s = np.linalg.lstsq(X_b, y, rcond=1e-6)
>>> theta_best_svd
array([[4.21509616],
       [2.77011339]])
```

This function computes $\widehat{\boldsymbol{\theta}} = \mathbf{X}^{+}\mathbf{y}$, where $\mathbf{X}^{+}$ is the *pseudoinverse* of $\mathbf{X}$ (specifically, the Moore-Penrose inverse). You can use `np.linalg.pinv()` to compute the pseudoinverse directly:

```
>>> np.linalg.pinv(X_b).dot(y)
array([[4.21509616],
       [2.77011339]])
```

Note: The normal equation may not work if the matrix X^TX is not invertible (i.e. singular), but the pseudo inverse is always defined.

Gradient Descent

Gradient Descent

Batch Gradient Descent

Stochastic Gradient Descent

Mini-batch Gradient Descent

Linear Regression – Gradient Descent

- Gradient descent is a generic optimization algorithm capable of finding optimal solutions to a wide range of problems.
- The general idea of gradient descent is to tweak parameters iteratively in order to minimize a cost function.
- Suppose you are lost in the mountains in a dense fog, and you can only feel the slope of the ground below your feet.
- A good strategy to get to the bottom of the valley quickly is to go downhill in the direction of the steepest slope.
- This is exactly what gradient descent does: it measures the local gradient of the error function with regard to the parameter vector θ, and it goes in the direction of descending gradient.
- Once the gradient is zero, you have reached a minimum!

Gradient Descent

- In practice, you start by filling θ with random values (this is called random initialization).
- Then you improve it gradually, taking one baby step at a time, each step attempting to decrease the cost function (e.g., the MSE), until the algorithm converges to a minimum (see Figure 4-3).

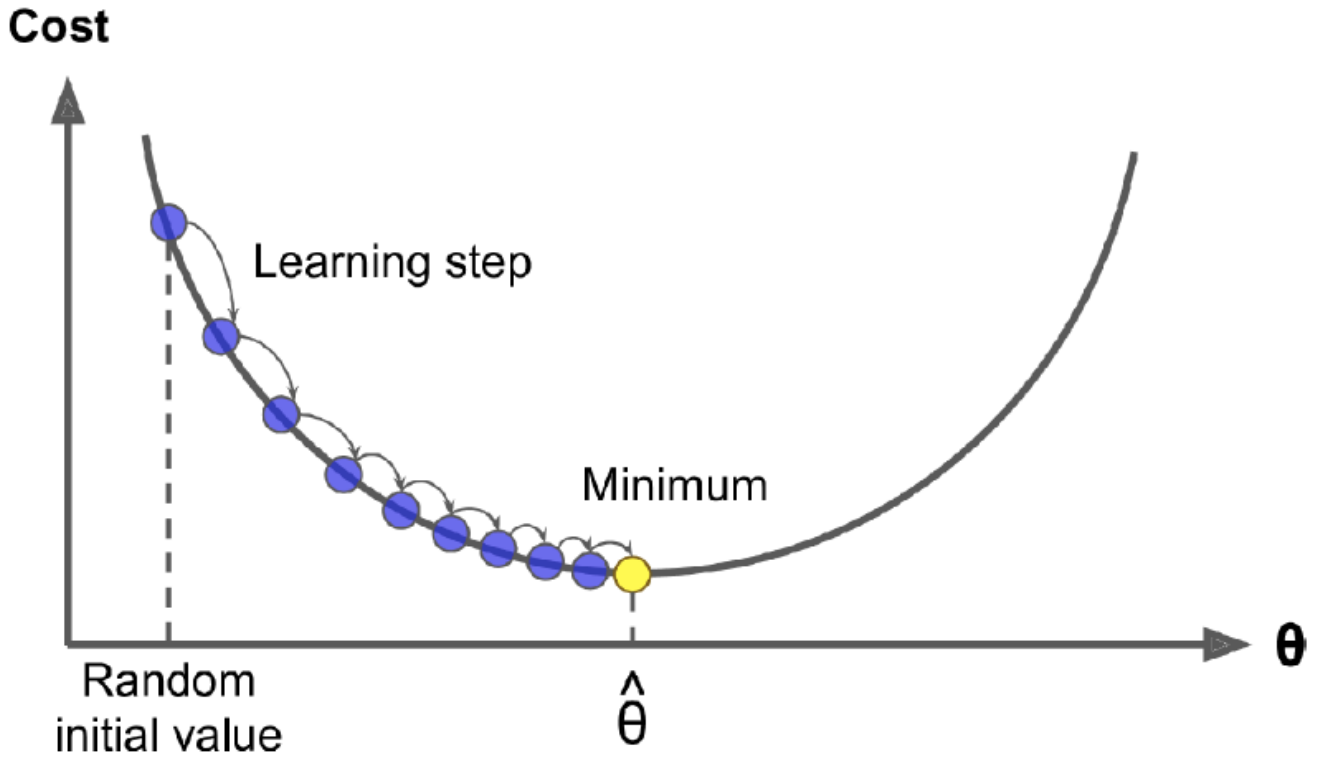


Figure 4-3. In this depiction of Gradient Descent, the model parameters are initialized randomly and get tweaked repeatedly to minimize the cost function; the learning step size is proportional to the slope of the cost function, so the steps gradually get smaller as the parameters approach the minimum

Gradient Descent – Learning Rate

- An important parameter in gradient descent is the size of the steps, determined by the learning rate hyperparameter.
- If **the learning rate is too small**, then the algorithm will have to go through many iterations to converge, which will take a long time (see Figure 4-4).

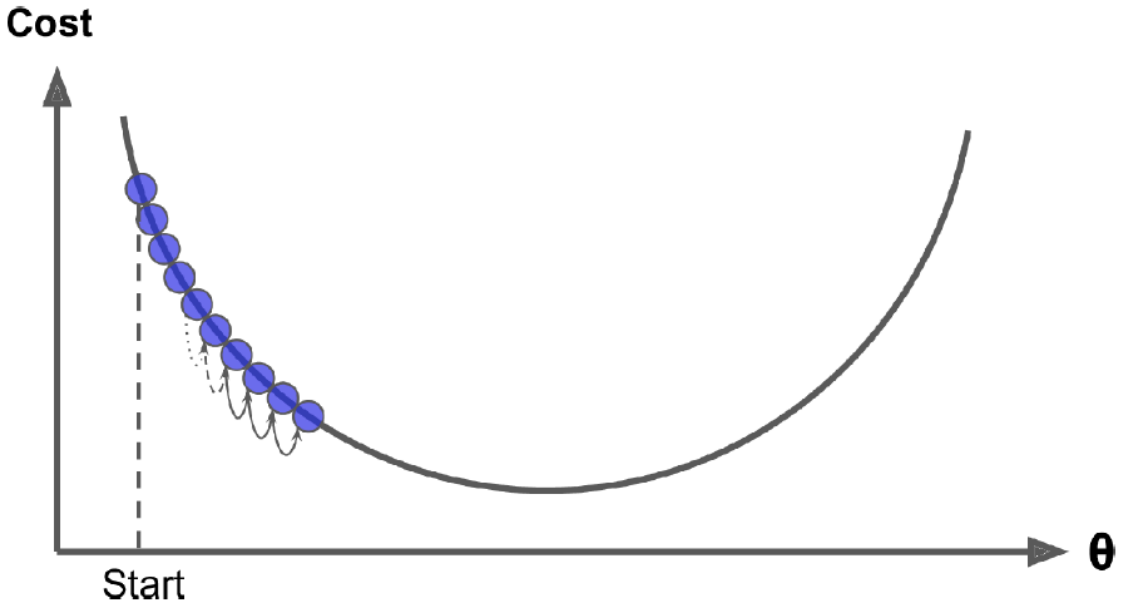


Figure 4-4. The learning rate is too small

Gradient Descent – Learning Rate

- On the other hand, if **the learning rate is too high**, you might jump across the valley and end up on the other side, possibly even higher up than you were before.
- This might make the algorithm diverge, with larger and larger values, failing to find a good solution (see Figure 4-5).

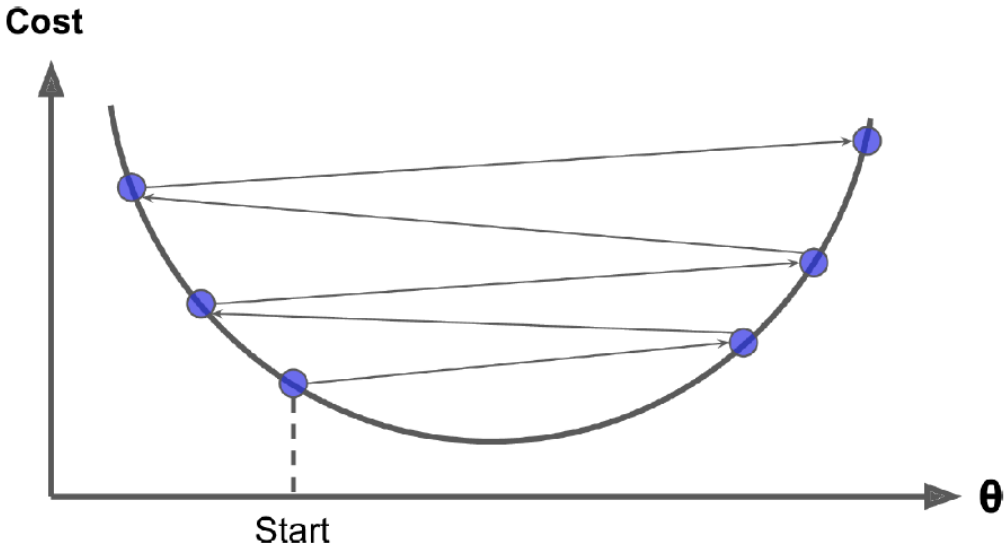


Figure 4-5. The learning rate is too large

Gradient Descent – Gradient Descent Pitfalls

- Not all cost functions look like nice, regular bowls.
- There may be holes, ridges, plateaus, and all sorts of irregular terrains, making convergence to the minimum difficult.
- If random initialization starts on the left, then it will converge to a local minimum, which is not as good as the global minimum.
- If it starts on the right, then it will take a long time to cross the plateau. And if you stop too early, you will never reach the global minimum.

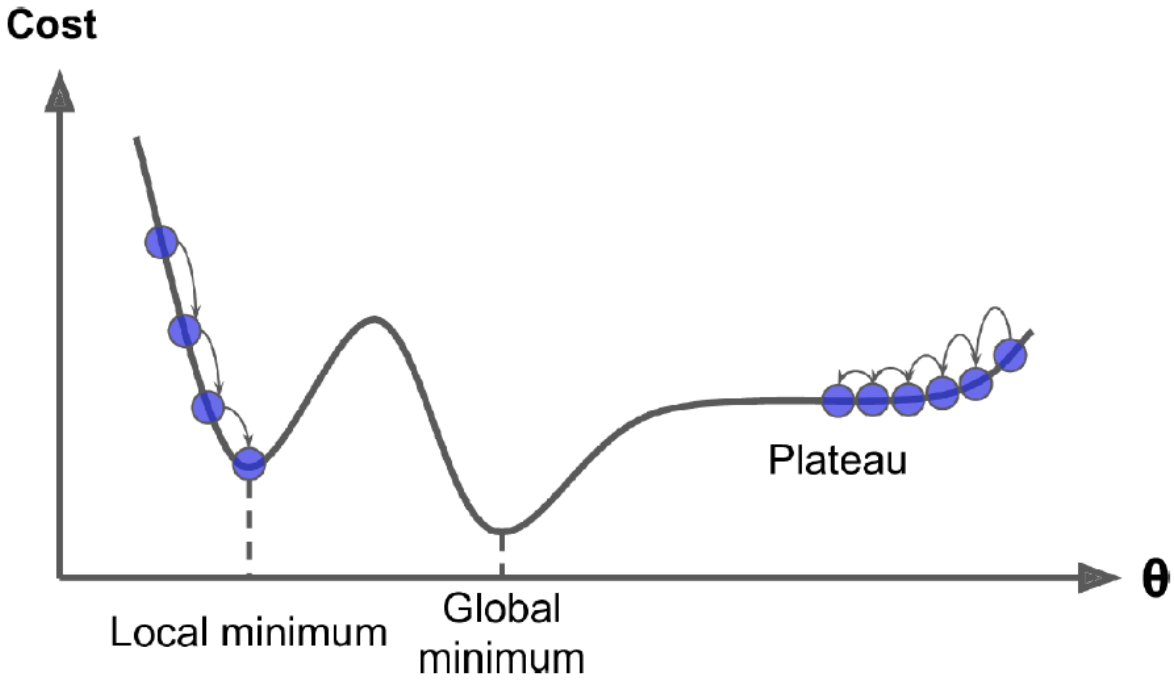


Figure 4-6. Gradient Descent pitfalls

Linear Regression – Gradient Descent

- Fortunately, the MSE cost function for Linear Regresion model happens to be a *convex function*. Hence, there are no local minima, just **one global minimum**.
- It is also a continuous function with a slope that never changes abruptly.
- Therefore, gradient descent is **guaranteed to approach arbitrarily close to the global minimum** (if you wait long enough and if the learning rate is not too high).

Linear Regression – Gradient Descent

- While the cost function has the shape of a bowl, it can be an elongated bowl if the features have very different scales.
- Figure 4-7 shows gradient descent on a training set where features 1 and 2 have the same scale (on the left), and on a training set where feature 1 has much smaller values than feature 2 (on the right).

-- As you can see, on the left the Gradient Descent algorithm goes straight toward the minimum, thereby reaching it quickly, whereas on the right it first goes in a direction almost orthogonal to the direction of the global minimum, and it ends with a long march down an almost flat valley.
-- It will eventually reach the minimum, but it will take a long time.

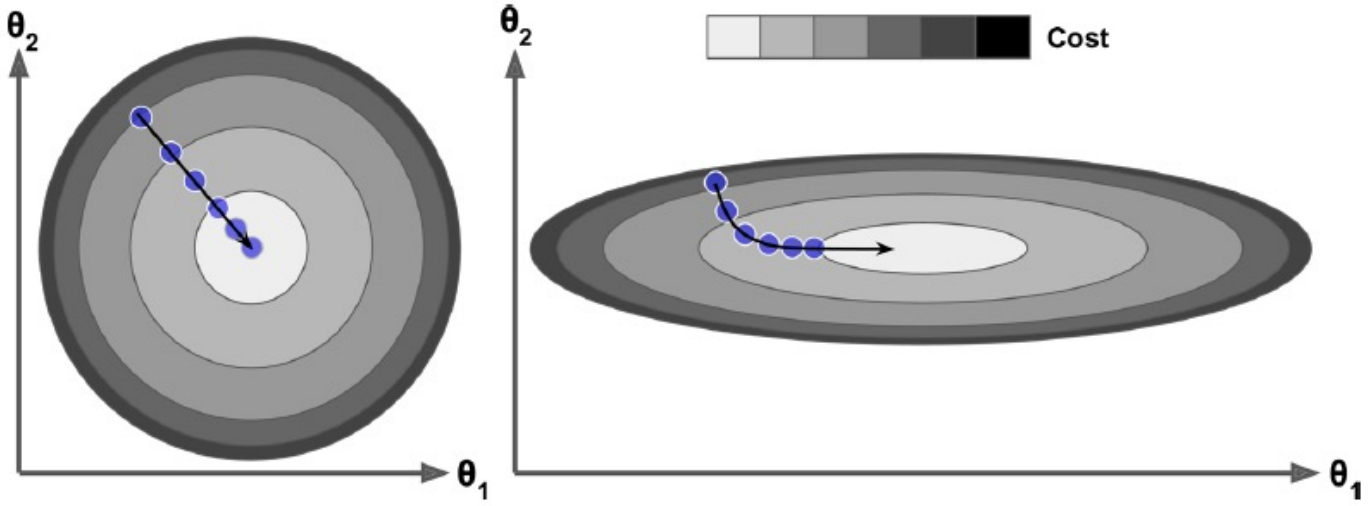


Figure 4-7. Gradient Descent with (left) and without (right) feature scaling

WARNING

When using Gradient Descent, you should ensure that all features have a similar scale (e.g., using Scikit-Learn's `StandardScaler` class), or else it will take much longer to converge.

Batch Gradient Descent

- To implement gradient descent, you need to compute the gradient of the cost function with regard to each model parameter θ_j.
- In other words, you need to calculate how much the cost function will change if you change θ_j just a little bit.
- This is called a **partial derivative**.
- Equation 4-5 computes the partial derivative of the MSE with regard to parameter θ_j, noted ∂ MSE(θ) / $\partial\theta_j$.

Equation 4-5. Partial derivatives of the cost function

$$\frac{\partial}{\partial \theta_j}\text{MSE}(\boldsymbol{\theta}) = \frac{2}{m}\sum_{i=1}^{m}\left(\boldsymbol{\theta}^{\intercal}\mathbf{x}^{(i)} - y^{(i)}\right)x_j^{(i)}$$

Batch Gradient Descent

- Instead of computing these partial derivatives individually, you can use Equation 4-6 to compute them all in one go.
- The gradient vector, noted $\nabla_\theta \mathrm{MSE}(\boldsymbol{\theta})$, contains all the partial derivatives of the cost function (one for each model parameter).

Equation 4-6. Gradient vector of the cost function

$$\nabla_{\boldsymbol{\theta}} \mathrm{MSE}(\boldsymbol{\theta}) = \begin{pmatrix} \frac{\partial}{\partial \theta_0} \mathrm{MSE}(\boldsymbol{\theta}) \\ \frac{\partial}{\partial \theta_1} \mathrm{MSE}(\boldsymbol{\theta}) \\ \vdots \\ \frac{\partial}{\partial \theta_n} \mathrm{MSE}(\boldsymbol{\theta}) \end{pmatrix} = \frac{2}{m} \mathbf{X}^\intercal (\mathbf{X}\boldsymbol{\theta} - \mathbf{y})$$

Batch Gradient Descent

- The formula involves calculations over the full training set **X**, at each Gradient Descent step!
- This is why the algorithm is called Batch Gradient Descent: it uses the whole batch of training data at every step.
- It is terribly slow on very large training datasets (we will see faster versions).
- Gradient descent scales well with the number of features: training linear regression model when there are hundreds of thousand of features is much faster using gradient descent than using the normal equation.

Batch Gradient Descent

- Once you have the gradient vector, which points uphill, just go in the opposite direction to go downhill.
- This means subtracting $\nabla_\theta \text{MSE}(\boldsymbol{\theta})$ from $\boldsymbol{\theta}$.
- This is where the learning rate η comes into play: multiply the gradient vector by η to determine the size of the downhill step (Equation 4-7).

Equation 4-7. Gradient descent step

$$\boldsymbol{\theta}^{(\text{next step})} = \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} \text{MSE}(\boldsymbol{\theta})$$

$$\boldsymbol{\theta}^{(\text{next step})} = \boldsymbol{\theta} - \eta \frac{2}{m} \mathbf{X}^\intercal (\mathbf{X}\boldsymbol{\theta} - \mathbf{y})$$

Batch Gradient Descent

Implementation

```
eta = 0.1  # learning rate
n_iterations = 1000
m = 100

theta = np.random.randn(2,1)  # random initialization

for iteration in range(n_iterations):
    gradients = 2/m * X_b.T.dot(X_b.dot(theta) - y)
    theta = theta - eta * gradients
```

That wasn't too hard! Let's look at the resulting `theta`:

```
>>> theta
array([[4.21509616],
       [2.77011339]])
```

Hey, that's exactly what the Normal Equation found! Gradient Descent worked perfectly. But what if you had used a different learning rate `eta`?

Batch Gradient Descent – Learning rate

- Figure 4-8 shows the first 20 steps of gradient descent using three different learning rates.
- The line at the bottom of each plot represents the random starting point, then each epoch is represented by a darker and darker line.
- On the left, the learning rate is too low: the algorithm will eventually reach the solution, but it will take a long time.
- In the middle, the learning rate looks pretty good: in just a few epochs, it has already converged to the solution.
- On the right, the learning rate is too high: the algorithm diverges, jumping all over the place and actually getting further and further away from the solution at every step.

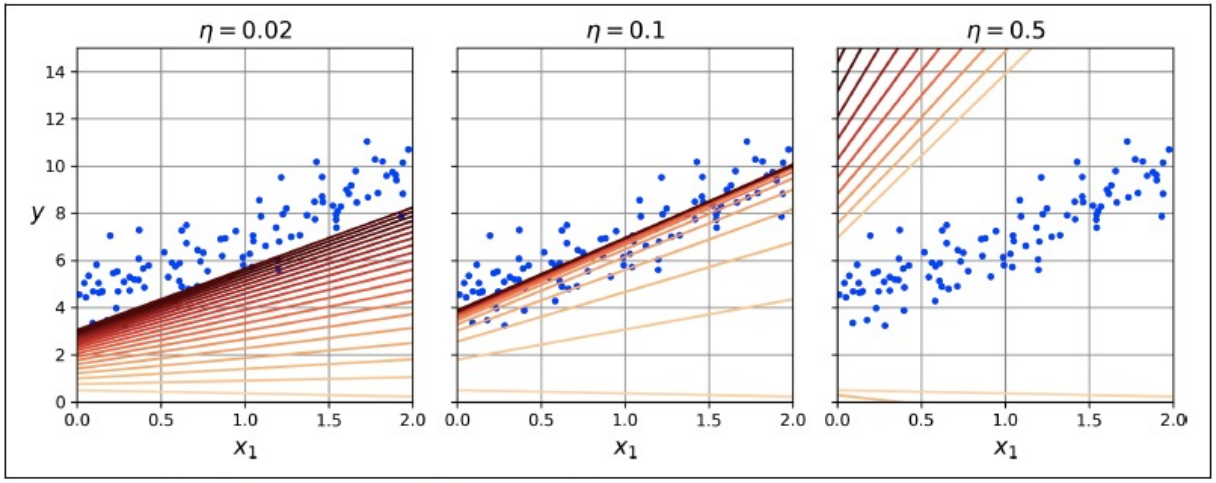


Figure 4-8. Gradient descent with various learning rates

Batch Gradient Descent – number of iterations

- You may wonder **how to set the number of epochs**.
- If it is too low, you will still be far away from the optimal solution when the algorithm stops; but if it is too high, you will waste time while the model parameters do not change anymore.
- A simple solution is to set a very large number of epochs but to interrupt the algorithm when the gradient vector becomes tiny—that is, when its norm becomes smaller than a tiny number ϵ (called the *tolerance*)—because this happens when gradient descent has (almost) reached the minimum.

Stochastic Gradient Descent - advantages

- The main problem with batch gradient descent is the fact that **it uses the whole training set to compute the gradients** at every step, which makes it very **slow when the training set is large**.
- At the opposite extreme, stochastic gradient descent picks a random instance in the training set at every step and computes the gradients based only on that single instance.
- Obviously, working on a single instance at a time makes the algorithm much faster because it has very little data to manipulate at every iteration.
- It also makes it possible to train on huge training sets, since only one instance needs to be in memory at each iteration.

Stochastic Gradient Descent - disadvantages

- On the other hand, due to its stochastic (i.e., random) nature, this algorithm is much less regular than batch gradient descent: instead of gently decreasing until it reaches the minimum, the cost function will bounce up and down, decreasing only on average.
- Over time it will end up very close to the minimum, but once it gets there it will continue to bounce around, never settling down (see Figure 4-9).
- Once the algorithm stops, the final parameter values will be good, but not optimal.

Stochastic Gradient Descent

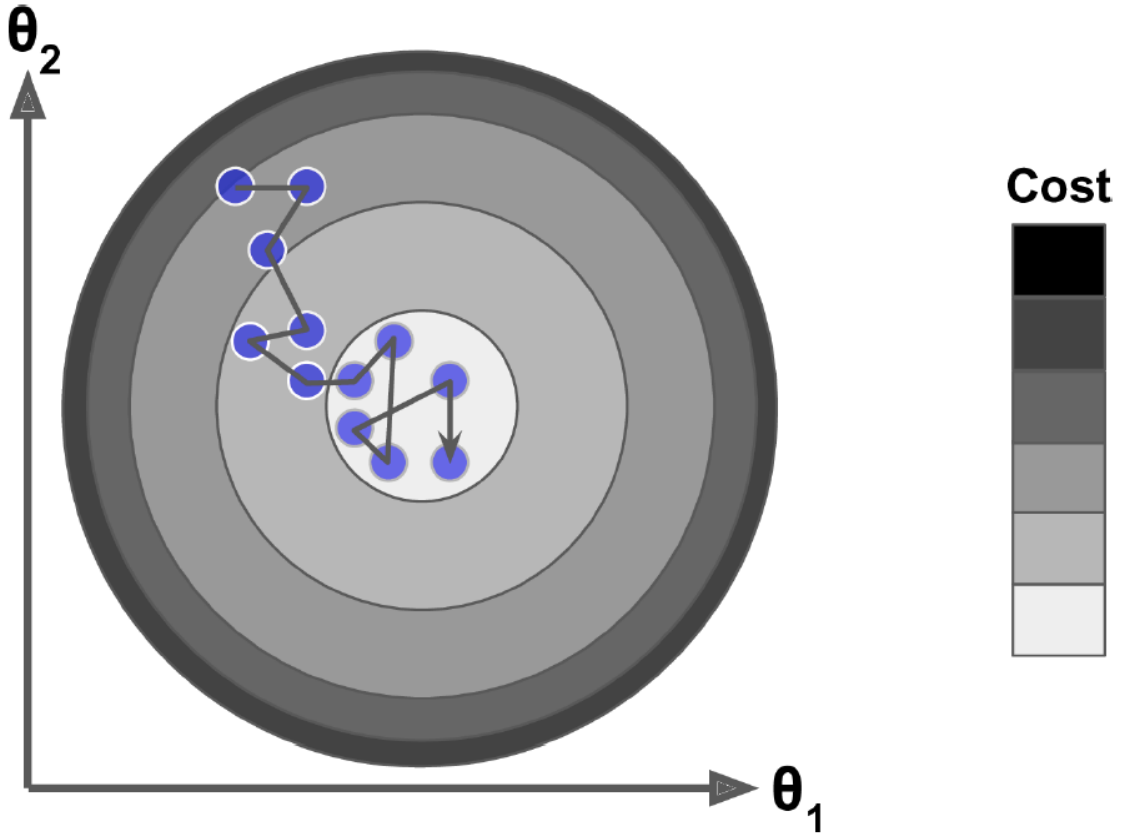


Figure 4-9. With Stochastic Gradient Descent, each training step is much faster but also much more stochastic than when using Batch Gradient Descent

Stochastic Gradient Descent

- When the cost function is very irregular (as in Figure 4-6), this can actually help the algorithm jump out of local minima, so **stochastic gradient descent has a better chance of finding the global minimum than batch gradient descent does**.
- Therefore, **randomness is good to escape from local optima**, but bad because it means that the algorithm can never settle at the minimum.
- One solution to this dilemma is to gradually reduce the learning rate. The steps start out large (which helps make quick progress and escape local minima), then get smaller and smaller, allowing the algorithm to settle at the global minimum.
- The function that determines the learning rate at each iteration is called the learning schedule.
- If the **learning rate is reduced too quickly**, you may **get stuck in a local minimum**, or even end up frozen halfway to the minimum.
- If the **learning rate is reduced too slowly**, you may **jump around the minimum for a long time** and end up with a suboptimal solution if you halt training too early.

Stochastic Gradient Descent: Implementation

- This code implements stochastic gradient descent using a simple learning schedule:
- By convention we iterate by rounds of m iterations; each round is called an epoch, as earlier.
- While the batch gradient descent code iterated 1,000 times through the whole training set, this code goes through the training set only 50 times and reaches a pretty good solution:

```
n_epochs = 50
t0, t1 = 5, 50  # learning schedule hyperparameters

def learning_schedule(t):
    return t0 / (t + t1)

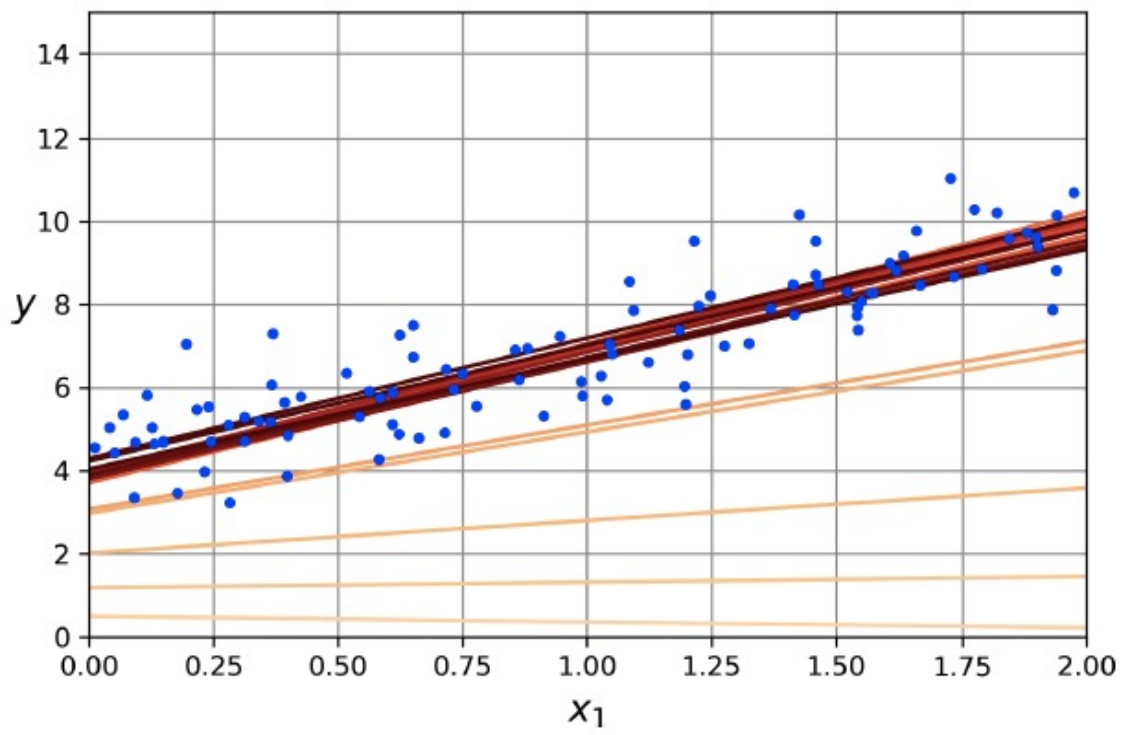
theta = np.random.randn(2,1)  # random initialization

for epoch in range(n_epochs):
    for i in range(m):
        random_index = np.random.randint(m)
        xi = X_b[random_index:random_index+1]
        yi = y[random_index:random_index+1]
        gradients = 2 * xi.T.dot(xi.dot(theta) - yi)
        eta = learning_schedule(epoch * m + i)
        theta = theta - eta * gradients
```

```
>>> theta
array([[4.21076011],
       [2.74856079]])
```

Stochastic Gradient Descent

- Notice how irregular the steps are.



0. The first 20 steps of stochastic gradient descent

Stochastic Gradient Descent
Implementation

- NOTE: Note that since instances are picked randomly, some instances may be picked several times per epoch, while others may not be picked at all.
- If you want to be sure that the algorithm goes through every instance at each epoch, another approach is to shuffle the training set (making sure to shuffle the input features and the labels jointly), then go through it instance by instance, then shuffle it again, and so on.
- However, this approach generally converges more slowly.

WARNING

When using Stochastic Gradient Descent, the training instances must be independent and identically distributed (IID) to ensure that the parameters get pulled toward the global optimum, on average. A simple way to ensure this is to shuffle the instances during training (e.g., pick each instance randomly, or shuffle the training set at the beginning of each epoch). If you do not shuffle the instances—for example, if the instances are sorted by label—then SGD will start by optimizing for one label, then the next, and so on, and it will not settle close to the global minimum.

Stochastic Gradient Descent
Implementation 2

- To perform linear regression using stochastic GD with Scikit-Learn, you can use the `SGDRegressor` class, which defaults to optimizing the MSE cost function.
- The following code runs for maximum 1,000 epochs (`max_iter`) or until the loss drops by less than 10–5 (`tol`) during 100 epochs (`n_iter_no_change`).
- It starts with a learning rate of 0.01 (`eta0`), using the default learning schedule (different from the one we used).
- Lastly, it does not use any regularization (`penalty=None`):

```
from sklearn.linear_model import SGDRegressor

sgd_reg = SGDRegressor(max_iter=1000, tol=1e-5, penalty=None, eta0=0.01,
                       n_iter_no_change=100, random_state=42)
sgd_reg.fit(X, y.ravel())  # y.ravel() because fit() expects 1D targets
```

```
>>> sgd_reg.intercept_, sgd_reg.coef_
(array([4.21278812]), array([2.77270267]))
```

Solution is quite close to true values (4,3).

Mini-batch Gradient Descent

Advantages, Disadvantages

- The last gradient descent algorithm we will look at is called mini-batch gradient descent.
- It is straightforward once you know batch and stochastic gradient descent: at each step, instead of computing the gradients based on the full training set (as in batch GD) or based on just one instance (as in stochastic GD), mini-batch GD computes the gradients on small random sets of instances called mini-batches.
- The main advantage of mini-batch GD over stochastic GD is that you can get a performance boost from hardware optimization of matrix operations, especially when using GPUs.

Mini-batch Gradient Descent

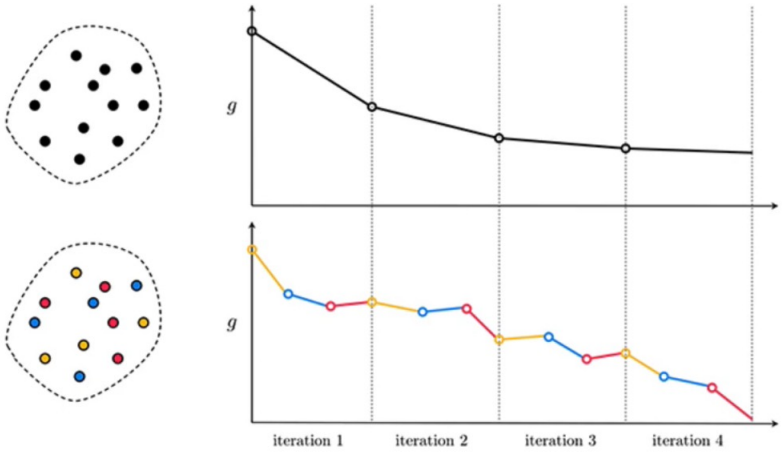
Advantages, Disadvantages

- The algorithm's progress in parameter space is less erratic than with stochastic GD, especially with fairly large mini-batches.
- As a result, mini-batch GD will end up walking around a bit closer to the minimum than stochastic GD—but it may be harder for it to escape from local minima (in the case of problems that suffer from local minima, unlike linear regression with the MSE cost function).

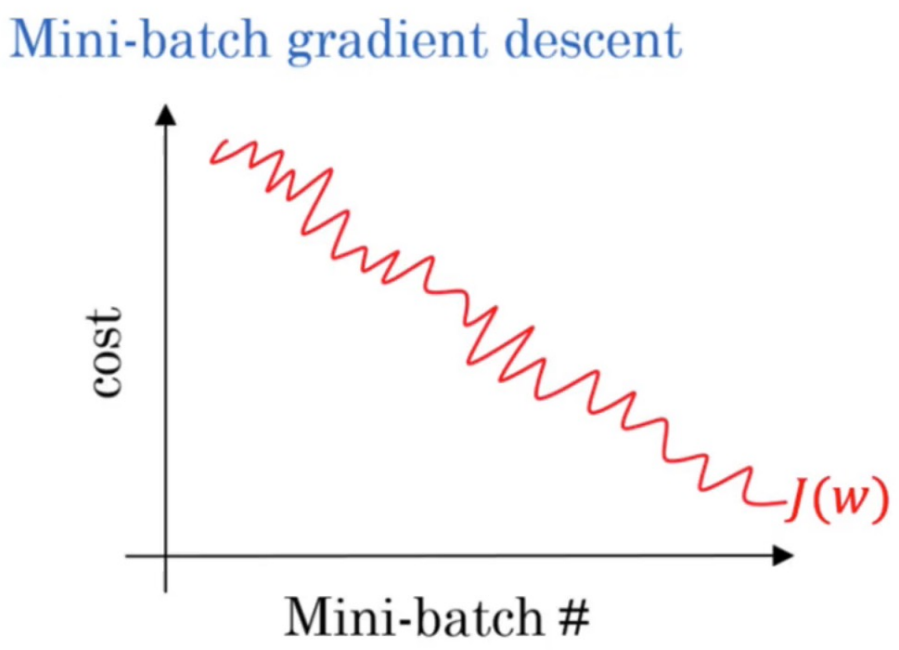
Mini-batch Gradient Descent

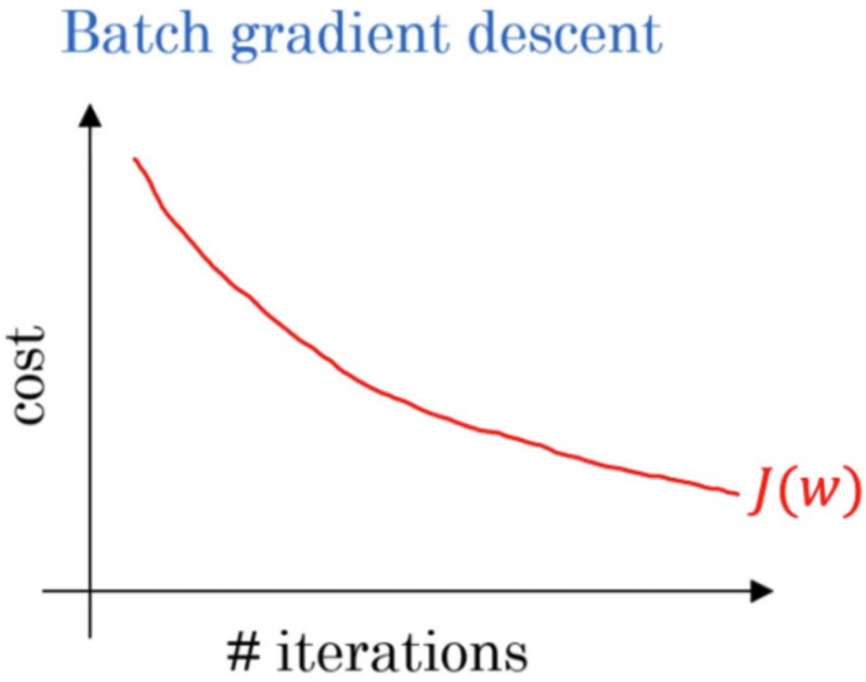
Mini-batch SGD introduces the concept of a batch size, *S*. Now, given a dataset of size *N*, there will be a total of *N* / *S* updates to the network:

1. Randomly shuffle the input data
2. Until convergence:
    1. Select the next batch of data of size *S*
    2. Make predictions on the subset
    3. Calculate the loss and mean gradient of the mini-batch
    4. Update the parameters of the network



Mini-batch Gradient Descent





Three Gradient Descent Algorithms

- Figure 4-11 shows the paths taken by the three gradient descent algorithms in parameter space during training.
- They all end up near the minimum, but batch GD's path actually stops at the minimum, while both stochastic GD and mini-batch GD continue to walk around.
- However, don't forget that batch GD takes a lot of time to take each step, and stochastic GD and mini-batch GD would also reach the minimum if you used a good learning schedule.

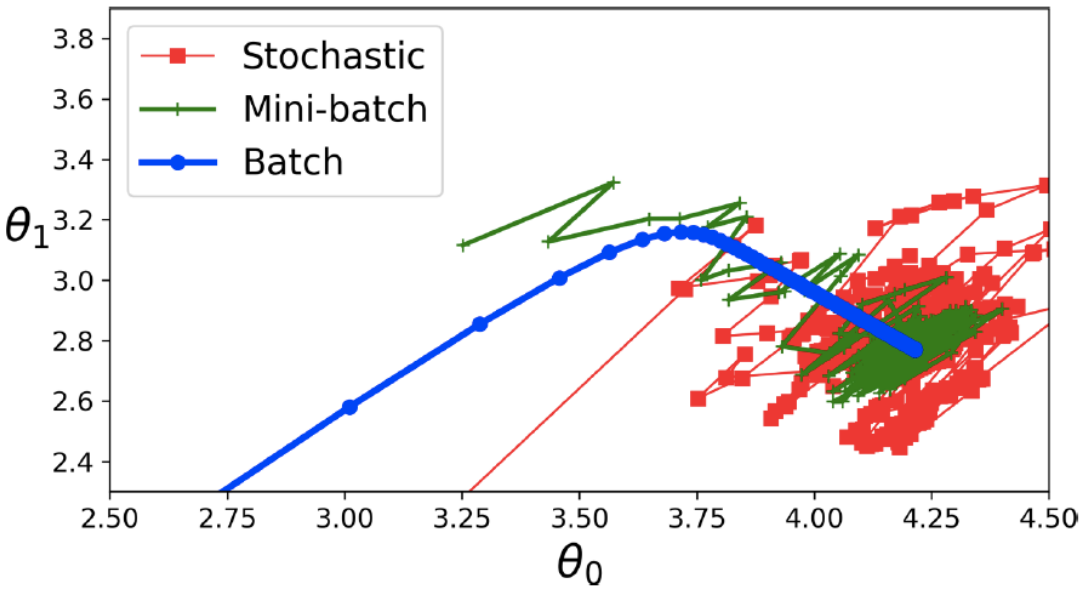


Figure 4-11. Gradient Descent paths in parameter space

Three Gradient Descent Algorithms

- There is almost no difference after training: all these algorithms end up with very similar models and make predictions in exactly the same way.

Table 4-1. Comparison of algorithms for linear regression

Algorithm	Large *m*	Out-of-core support	Large *n*	Hyperparams	Scaling required	Scikit-Learn
Normal equation	Fast	No	Slow	0	No	N/A
SVD	Fast	No	Slow	0	No	`LinearRegression`
Batch GD	Slow	No	Fast	2	Yes	N/A
Stochastic GD	Fast	Yes	Fast	≥2	Yes	`SGDRegressor`
Mini-batch GD	Fast	Yes	Fast	≥2	Yes	N/A

Polynomial Regression

Polynomial Regression

- What if the data is more complex than a line?
- We can use a linear model to fit nonlinear data
- A simple way: add powers of each feature as new features, then train a linear model on this extended set of features.
- This technique is called *polynomial regression*.

Polynomial Regression - Example

- Quadratic equation, plus some noise:

```
m = 100
X = 6 * np.random.rand(m, 1) - 3
y = 0.5 * X**2 + X + 2 + np.random.randn(m, 1)
```

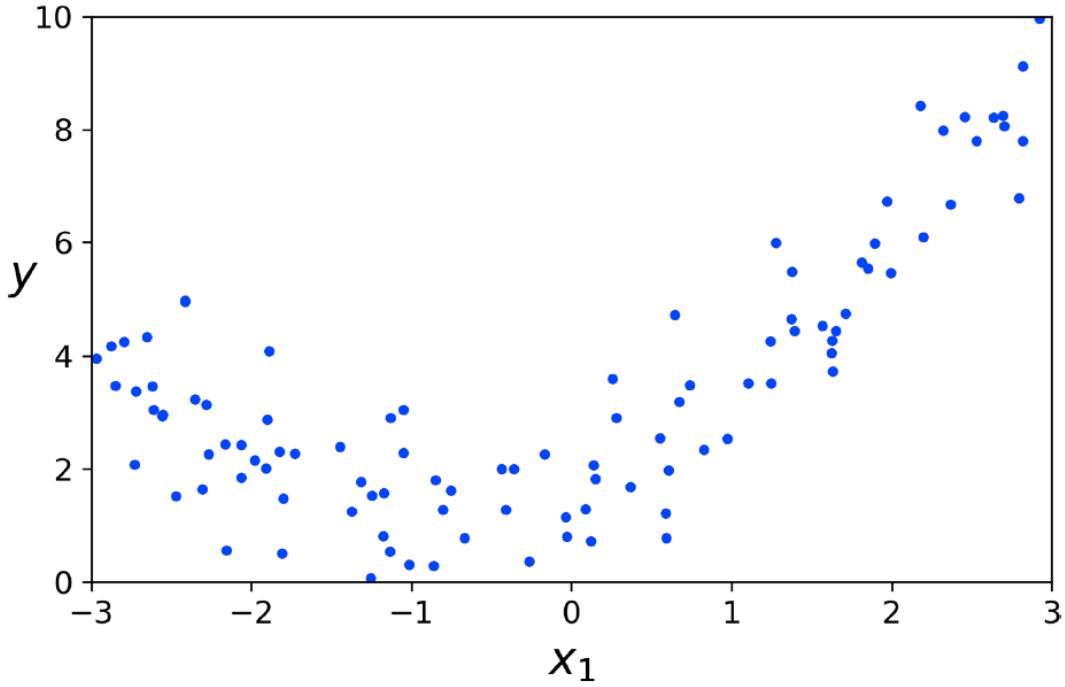


Figure 4-12. Generated nonlinear and noisy dataset

Polynomial Regression - Example

- Clearly, a straight line will never fit this data properly.
- So let's use Scikit-Learn's `PolynomialFeatures` class to transform our training data, adding the square (second degree polynomial) of each feature in the training set as a new feature (in this case there is just one feature):

```
>>> from sklearn.preprocessing import PolynomialFeatures
>>> poly_features = PolynomialFeatures(degree=2, include_bias=False)
>>> X_poly = poly_features.fit_transform(X)
>>> X[0]
array([-0.75275929])
>>> X_poly[0]
array([-0.75275929,  0.56664654])
```

- `X_poly` now contains the original feature of `X` plus the square of this feature.

Polynomial Regression - Example

Now you can fit a `LinearRegression` model to this extended training data (Figure 4-13):

```
>>> lin_reg = LinearRegression()
>>> lin_reg.fit(X_poly, y)
>>> lin_reg.intercept_, lin_reg.coef_
(array([1.78134581]), array([[0.93366893, 0.56456263]]))
```

Not bad: the model estimates $\hat{y} = 0.56{x_1}^2 + 0.93x_1 + 1.78$ when in fact the original function was $y = 0.5{x_1}^2 + 1.0x_1 + 2.0 + \text{Gaussian noise}$.

Polynomial Regression - Example

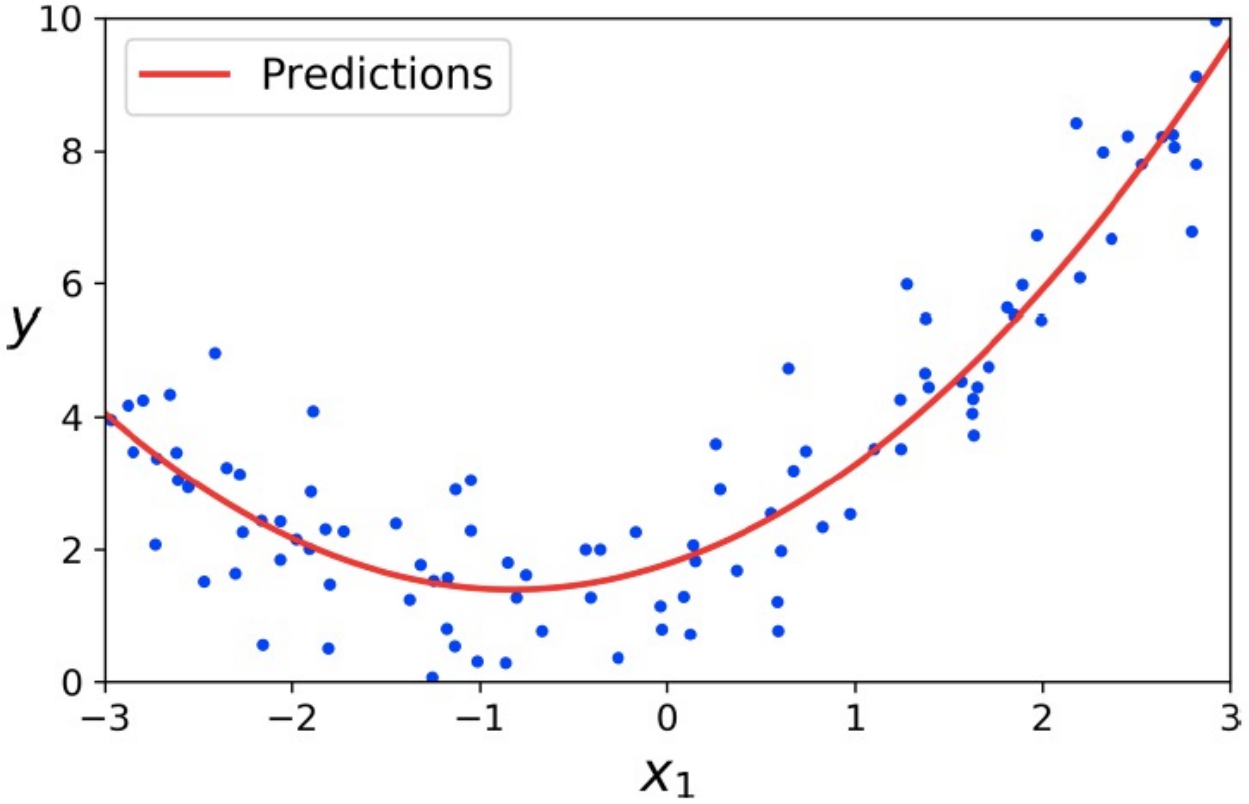


Figure 4-13. Polynomial Regression model predictions

Polynomial Regression - Example

- Note that when there are multiple features, polynomial regression is capable of finding relationships between features, which is something a plain linear regression model cannot do.
- This is made possible by the fact that `PolynomialFeatures` also adds all combinations of features up to the given degree.
- For example, if there were two features a and b, `PolynomialFeatures` with `degree=3` would not only add the features a^2, a^3, b^2, and b^3, but also the combinations ab, a^2b, and ab^2.

Polynomial Regression - Example

WARNING

`PolynomialFeatures(degree=d)` transforms an array containing n features into an array containing $\frac{(n+d)!}{d!\,n!}$ features, where $n!$ is the *factorial* of n, equal to $1 \times 2 \times 3 \times \cdots \times n$. Beware of the combinatorial explosion of the number of features!

Learning Curves

- If you perform high-degree Polynomial Regression, you will likely fit the training data much better than with plain Linear Regression.
- For example, Figure 4-14 applies a 300-degree polynomial model to the preceding training data, and compares the result with a pure linear model and a quadratic model (second-degree polynomial).
- Notice how the 300-degree polynomial model wiggles around to get as close as possible to the training instances.

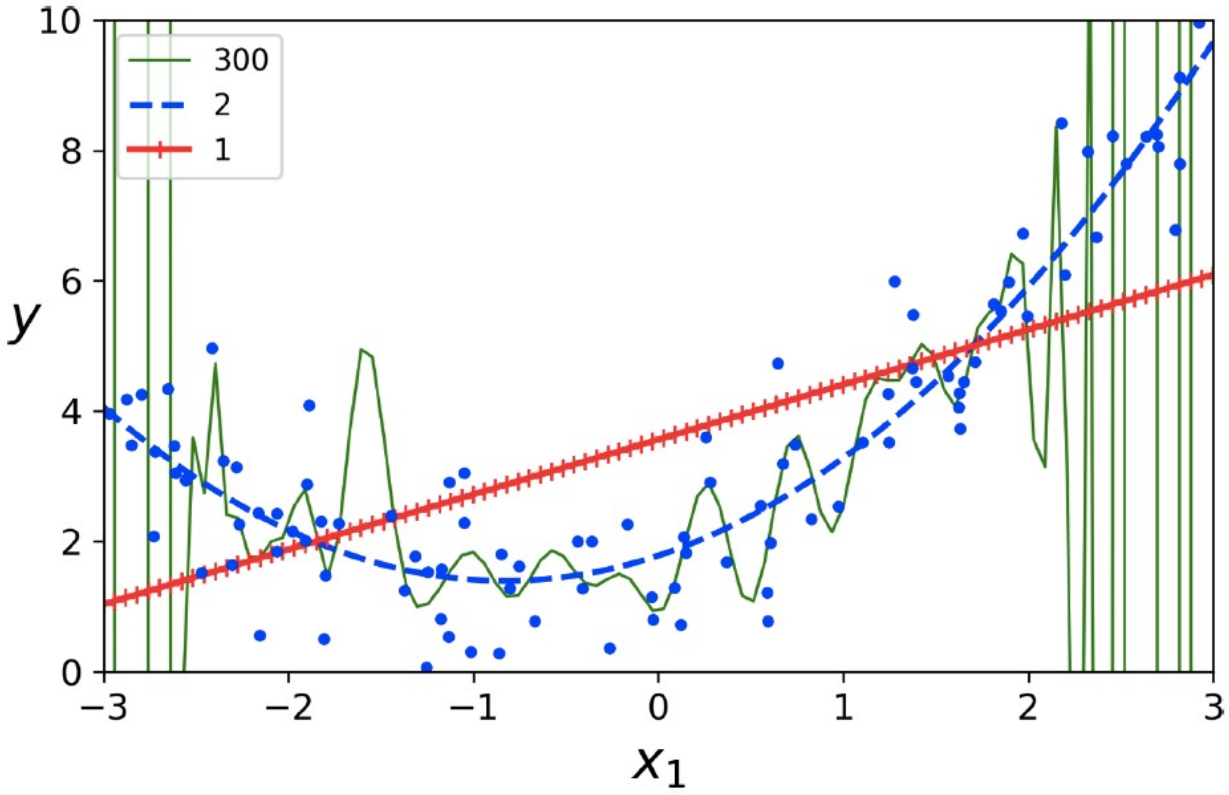


Figure 4-14. High-degree Polynomial Regression

- This high-degree Polynomial Regression model is severely overfitting the training data, while the linear model is underfitting it.
- The model that will generalize best in this case is the quadratic model, which makes sense because the data was generated using a quadratic model.
- But in general you won't know what function generated the data, so how can you decide how complex your model should be?
- How can you tell that your model is overfitting or underfitting the data?

- Cross-validation gives an estimate of a model's generalization performance:
  - If a model performs well on the training data but generalizes poorly according to the cross-validation metrics, then the model is overfitting.
  - If it performs poorly on both, then it is underfitting.
- This is one way to tell when a model is too simple or too complex.

Learning Curves

- Another way to tell is to look at learning curves.
- Learning curves: Plots of the model's performance on the training set and the validation set as a function of the training set size.

Learning Curves

Code

```
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split

def plot_learning_curves(model, X, y):
    X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2)
    train_errors, val_errors = [], []
    for m in range(1, len(X_train)):
        model.fit(X_train[:m], y_train[:m])
        y_train_predict = model.predict(X_train[:m])
        y_val_predict = model.predict(X_val)
        train_errors.append(mean_squared_error(y_train[:m],
y_train_predict))
        val_errors.append(mean_squared_error(y_val, y_val_predict))
    plt.plot(np.sqrt(train_errors), "r-+", linewidth=2, label="train")
    plt.plot(np.sqrt(val_errors), "b-", linewidth=3, label="val")
```

Let's look at the learning curves of the plain Linear Regression model (a straight line; see Figure 4-15):

```
lin_reg = LinearRegression()
plot_learning_curves(lin_reg, X, y)
```

Learning Curves

- These learning curves are typical of a model that's underfitting.
- Both curves have reached a plateau; they are close and fairly high.

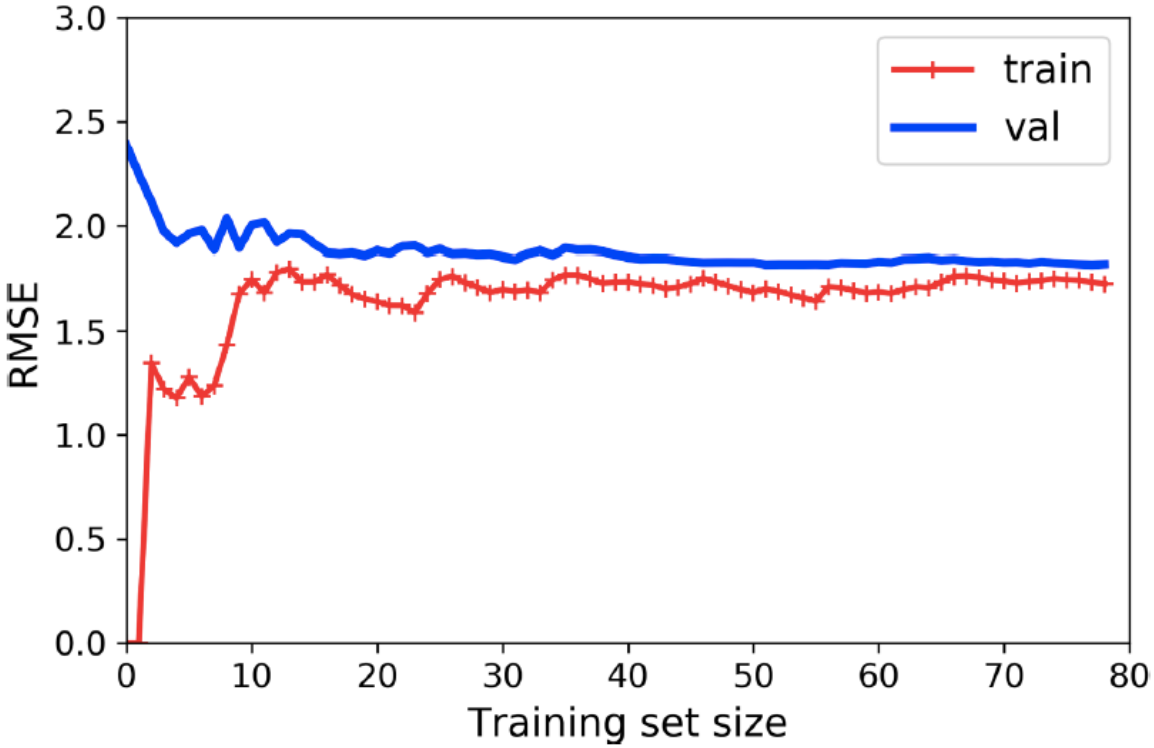


Figure 4-15. Learning curves

Learning Curves

- This model that's underfitting deserves a bit of explanation.
- First, let's look at the performance on the training data:
  - when there are just one or two instances in the training set, the model can fit them perfectly, which is why the curve starts at zero.
  - But as new instances are added to the training set, it becomes impossible for the model to fit the training data perfectly, both because the data is noisy and because it is not linear at all.
  - So the error on the training data goes up until it reaches a plateau, at which point adding new instances to the training set doesn't make the average error much better or worse.
- Now let's look at the performance of the model on the validation data.
  - When the model is trained on very few training instances, it is incapable of generalizing properly, which is why the validation error is initially quite big.
  - Then, as the model is shown more training examples, it learns, and thus the validation error slowly goes down.
  - However, once again a straight line cannot do a good job modeling the data, so the error ends up at a plateau, very close to the other curve.



TIP

If your model is underfitting the training data, adding more training examples will not help. You need to use a more complex model or come up with better features.

Learning Curves
10-th Degree Polynomial

- Now let's look at the learning curves of a 10th-degree polynomial model on the same data (Figure 4-16):

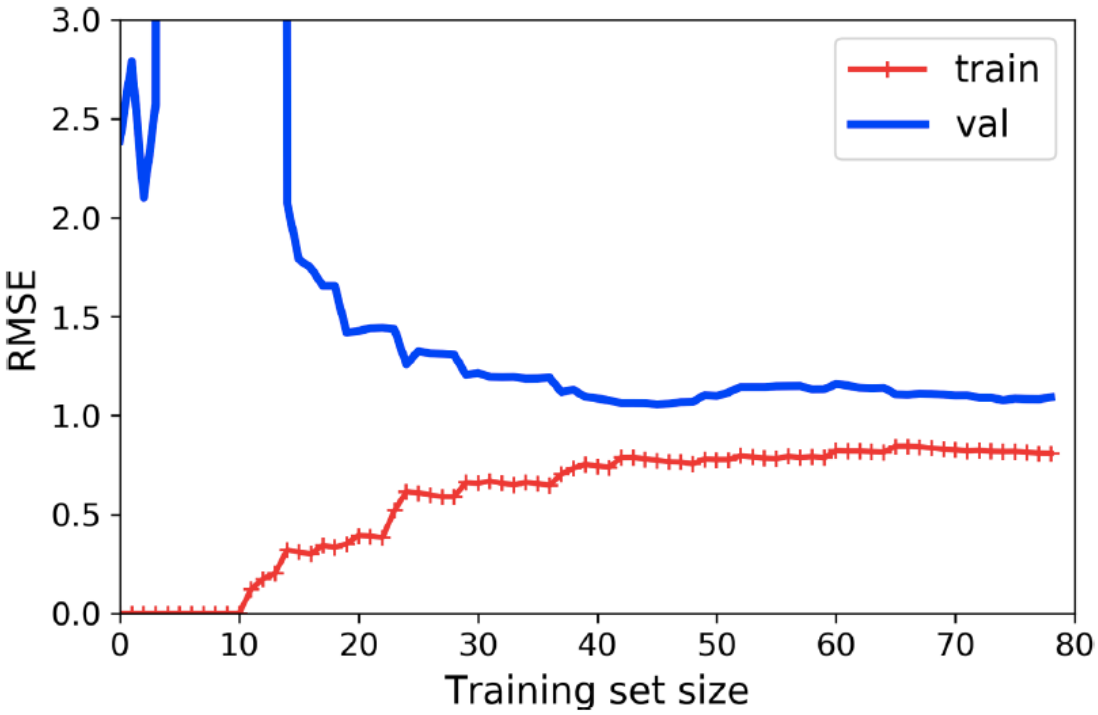


Figure 4-16. Learning curves for the 10th-degree polynomial model

```
from sklearn.pipeline import Pipeline

polynomial_regression = Pipeline([
        ("poly_features", PolynomialFeatures(degree=10,
include_bias=False)),
        ("lin_reg", LinearRegression()),
    ])

plot_learning_curves(polynomial_regression, X, y)
```

- The error on the training data is much lower than with the Linear Regression model.
- There is a gap between the curves. This means that the model performs significantly better on the training data than on the validation data, which is the hallmark of an overfitting model.
- If you used a much larger training set, however, the two curves would continue to get closer.



TIP

One way to improve an overfitting model is to feed it more training data until the validation error reaches the training error.

Bias-Variance Trade-Off

- An important theoretical result of statistics and machine learning is the fact that a model's generalization error can be expressed as the sum of three very different errors:
- Bias: This part of the generalization error is due to wrong assumptions, such as assuming that the data is linear when it is actually quadratic. A high-bias model is most likely to underfit the training data.
- Variance: This part is due to the model's excessive sensitivity to small variations in the training data. A model with many degrees of freedom (such as a high-degree polynomial model) is likely to have high variance and thus overfit the training data.
- Irreducible error: This part is due to the noisiness of the data itself. The only way to reduce this part of the error is to clean up the data (e.g., fix the data sources, such as broken sensors, or detect and remove outliers).
- Increasing a model's complexity will typically increase its variance and reduce its bias. Conversely, reducing a model's complexity increases its bias and reduces its variance. This is why **it is called a trade-off**.

Regularized Linear Models

Ridge Regression (L2 Regularization)

Lasso Regression

Elastic Net

Early Stopping

Regularized Linear Models

- A good way to reduce overfitting is to regularize the model (i.e. constrain it).
- A simple way to regularize a polynomial model is to reduce the number of polynomial degrees.
- For a linear model regularization is typically achieved by constraining the weights of the model.

Ridge Regression (L2 Regularization)

- Also called as *Tikhonov regularization*
- A *regularization term* based on L_2 norm of the parameter vector is added to the cost function to force the learning algorithm to not only fit the data but also keep the model weights as small as possible.
- The regularization term is **only added during training**. Once trained, use the unregularized performance measure to evaluate the model's performance.

Equation 4-8. Ridge Regression cost function

$$J(\boldsymbol{\theta}) = \text{MSE}(\boldsymbol{\theta}) + \alpha \frac{1}{2} \sum_{i=1}^{n} {\theta_i}^2$$

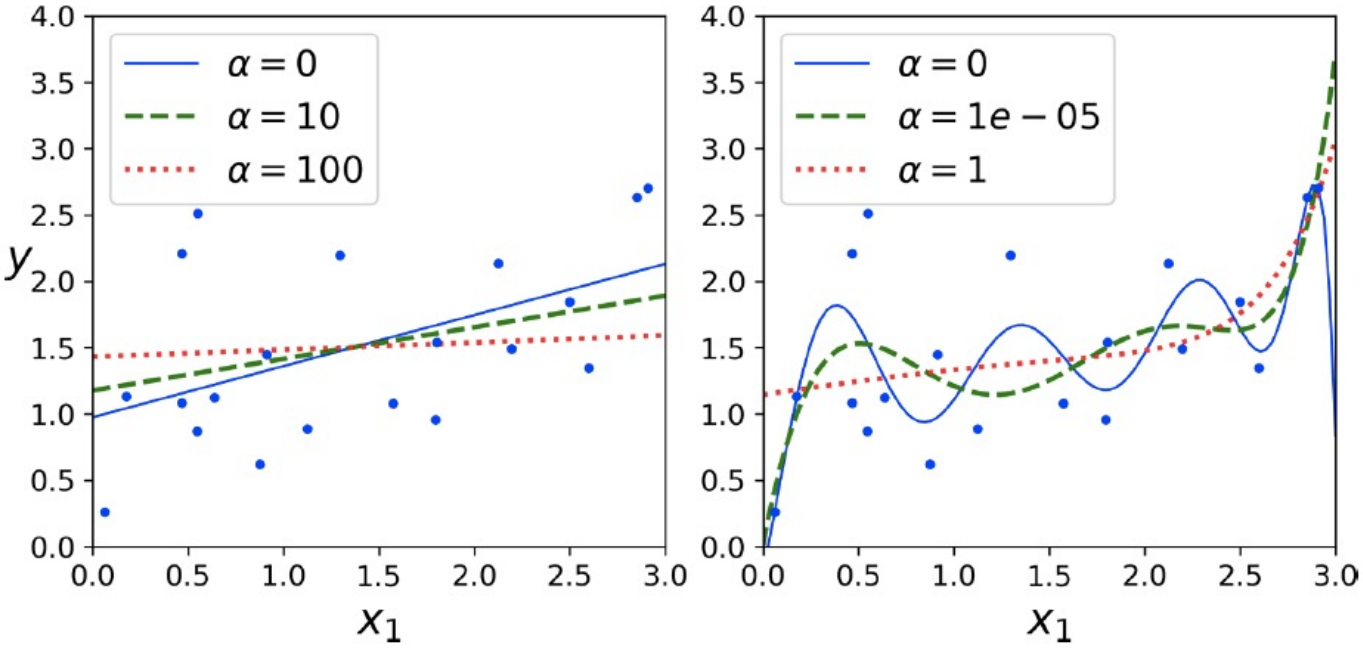
Note that the bias term θ_0 is not regularized (the sum starts at $i = 1$, not 0).

WARNING

It is important to scale the data (e.g., using a `StandardScaler`) before performing Ridge Regression, as it is sensitive to the scale of the input features. This is true of most regularized models.

Ridge Regression - Results

- Figure 4-17 shows several Ridge models trained on some linear data using different α values.
- On the left, plain Ridge models are used, leading to linear predictions.
- On the right, the data is first expanded using `PolynomialFeatures(degree=10)`, then it is scaled using a `StandardScaler`, and finally the Ridge models are applied to the resulting features: this is Polynomial Regression with Ridge regularization.
- Note how increasing α leads to flatter (i.e., less extreme, more reasonable) predictions, thus reducing the model's variance but increasing its bias.



Ridge Regression
Implementation

- As with Linear Regression, we can perform Ridge Regression either by computing a closed-form equation or by performing Gradient Descent. The pros and cons are the same.
- Equation below shows the closed-form solution, where A is the (n + 1) × (n + 1) identity matrix, except with a 0 in the topleft cell, corresponding to the bias term.

Equation 4-9. Ridge Regression closed-form solution

$$\widehat{\boldsymbol{\theta}} = (\mathbf{X}^\intercal \mathbf{X} + \alpha \mathbf{A})^{-1} \mathbf{X}^\intercal \mathbf{y}$$

Ridge Regression
Implementation

- How to perform it with Scikit-Learn using a closed-form solution:

```
>>> from sklearn.linear_model import Ridge
>>> ridge_reg = Ridge(alpha=1, solver="cholesky")
>>> ridge_reg.fit(X, y)
>>> ridge_reg.predict([[1.5]])
array([[1.55071465]])
```

- Using Stochastic Gradient Descent:

```
>>> sgd_reg = SGDRegressor(penalty="l2")
>>> sgd_reg.fit(X, y.ravel())
>>> sgd_reg.predict([[1.5]])
array([1.47012588])
```

L_2 norm indicates ridge regression

A 1-D array, containing the elements of y, is returned

Lasso Regression (L1 Regularization)

- *Least Absolute Shinkage and Selection Operator* (LASSO) regression adds a regularization term to the cost function using L_1 norm of the weight vector:
- Cost fuction:

$$J(\boldsymbol{\theta}) = \text{MSE}(\boldsymbol{\theta}) + \alpha \sum_{i=1}^{n} |\theta_i|$$

- Important characteristic of Lasso regression:
  - Eliminates the weights of least important features (i.e. sets them to zero). In other words, Lasso regression automatically performs feature selection and outputs a sparse model (i.e. with few non-zero feature weights)

Lasso Regression - Example

- Model parameters are initialized as:
- Left: $\theta_1 = 2$, $\theta_2 = 0.5$
- Right:
  - $\theta_1 = 0.25$, $\theta_2 = -1$
  - Optimal parameters for the unregularized MSE are $\theta_1 = 2$ and $\theta_2 = 0.5$

notice once again how the path quickly reaches $\theta_2 = 0$,

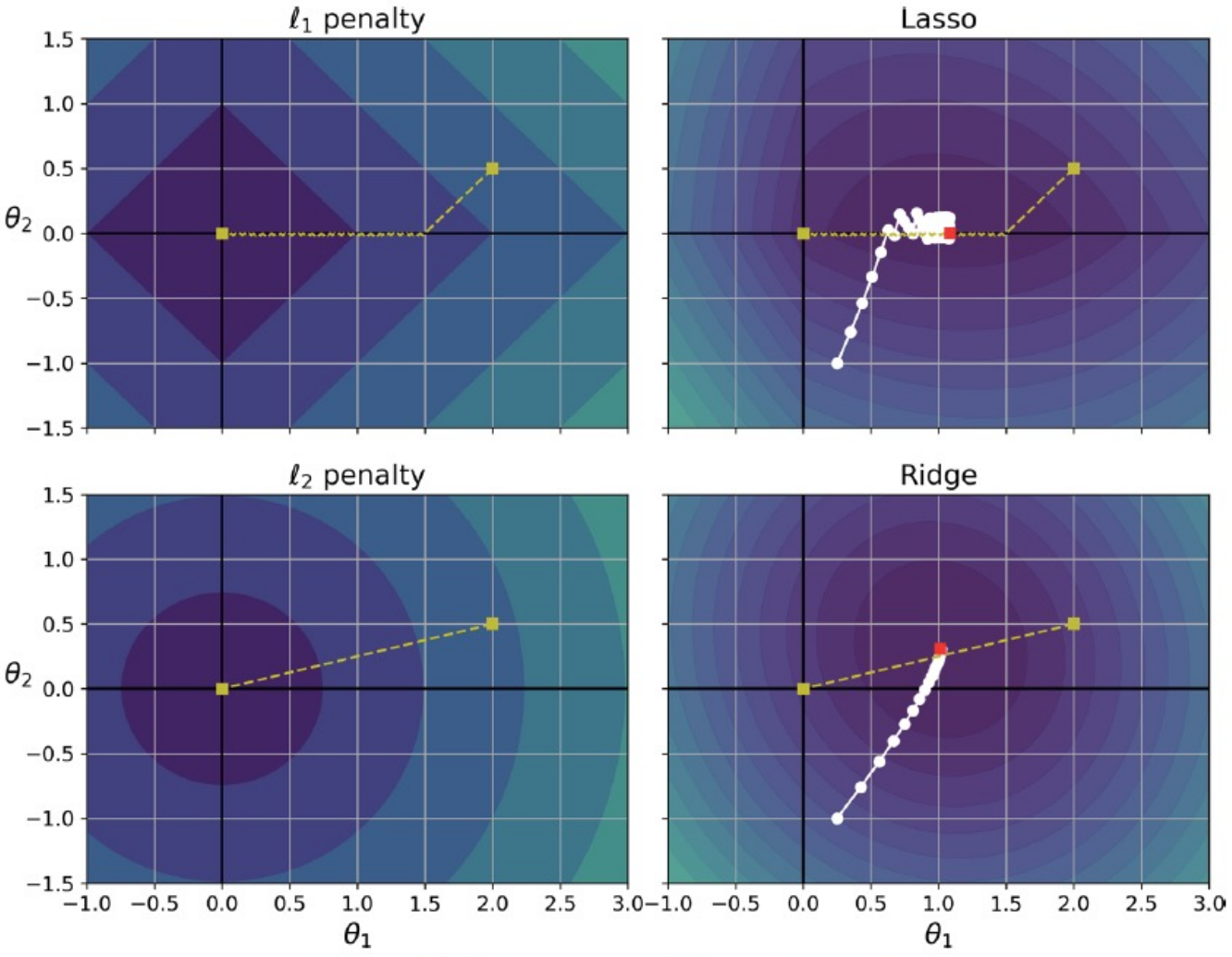


Figure 4-19. Lasso versus Ridge regularization

Lasso Regression - Example

- You can get a sense of why this is the case by looking at Figure 4-19: the axes represent two model parameters, and the background contours represent different loss functions.
- In the **top-left plot**, the contours represent the ℓ_1 Loss ($|\theta_1| + |\theta_2|$), which drops linearly as you get closer to any axis.
  - For example, if you **initialize the model parameters to $\theta_1 = 2$ and $\theta_2 = 0.5$**, running Gradient Descent will decrement both parameters equally (as represented by the dashed yellow line); therefore θ_2 will reach 0 first (since it was closer to 0 to begin with).
  - After that, Gradient Descent will roll down the axis until it reaches $\theta_1 = 0$ ((with a bit of bouncing around, since the gradients of ℓ_1 never get close to 0: they are either –1 or 1 for each parameter).

Lasso Regression - Example

- In the **top right plot** of Figure 4-19, the contours represent lasso regression's cost function (i.e., an MSE cost function plus an ℓ_1 loss).
  - The small white circles show the path that gradient descent takes to optimize some model parameters that were **initialized around $\theta_1 = 0.25$ and $\theta_2 = -1$**: notice once again how the path quickly reaches $\theta_2 = 0$, then rolls down the gutter and ends up bouncing around the global optimum (represented by the red square).
  - If we increased α, the global optimum would move left along the dashed yellow line, while if we decreased α, the global optimum would move right (in this example, the optimal parameters for the unregularized MSE are $\theta_1 = 2$ and $\theta_2 = 0.5$).

Lasso Regression - Example

- The two bottom plots show the same thing but with an ℓ_2 penalty instead.
  - In the **bottom-left plot**, you can see that the ℓ_2 loss decreases as we get closer to the origin, so gradient descent just takes a straight path toward that point.
  - In the **bottom-right plot**, the contours represent ridge regression's cost function (i.e., an MSE cost function plus an ℓ_2 loss).
    - As you can see, the gradients get smaller as the parameters approach the global optimum, so gradient descent naturally slows down.
    - This limits the bouncing around, which helps **ridge converge faster than lasso regression**.
    - Also note that the optimal parameters (represented by the red square) get closer and closer to the origin when you increase α, but they never get eliminated entirely.

Lasso Regression

TIP

To avoid Gradient Descent from bouncing around the optimum at the end when using Lasso, you need to gradually reduce the learning rate during training (it will still bounce around the optimum, but the steps will get smaller and smaller, so it will converge).

Lasso Regression

The Lasso cost function is not differentiable at $\theta_i = 0$ (for $i = 1, 2, \cdots, n$), but Gradient Descent still works fine if you use a *subgradient vector* $\mathbf{g}$[13] instead when any $\theta_i = 0$. Equation 4-11 shows a subgradient vector equation you can use for Gradient Descent with the Lasso cost function.

Equation 4-11. Lasso Regression subgradient vector

$$g(\boldsymbol{\theta}, J) = \nabla_{\boldsymbol{\theta}} \operatorname{MSE}(\boldsymbol{\theta}) + \alpha \begin{pmatrix} \operatorname{sign}(\theta_1) \\ \operatorname{sign}(\theta_2) \\ \vdots \\ \operatorname{sign}(\theta_n) \end{pmatrix} \quad \text{where } \operatorname{sign}(\theta_i) = \begin{cases} -1 & \text{if } \theta_i < 0 \\ 0 & \text{if } \theta_i = 0 \\ +1 & \text{if } \theta_i > 0 \end{cases}$$

Lasso Regression

Implementation

Here is a small Scikit-Learn example using the `Lasso` class:

```
>>> from sklearn.linear_model import Lasso
>>> lasso_reg = Lasso(alpha=0.1)
>>> lasso_reg.fit(X, y)
>>> lasso_reg.predict([[1.5]])
array([1.53788174])
```

Note that you could instead use `SGDRegressor(penalty="l1")`.

Elastic Nets

- Elastic Net is a middle ground between Ridge Regression and Lasso Regression.
- The regularization term is a simple mix of both Ridge and Lasso's regularization terms, and you can control the mix ratio r. When $r = 0$, Elastic Net is equivalent to Ridge Regression, and when $r = 1$, it is equivalent to Lasso Regression (see Equation 4-12).

Equation 4-12. Elastic Net cost function

$$J(\boldsymbol{\theta}) = \text{MSE}(\boldsymbol{\theta}) + r\alpha \sum_{i=1}^{n} |\theta_i| + \frac{1-r}{2} \alpha \sum_{i=1}^{n} {\theta_i}^2$$

Here is a short example that uses Scikit-Learn's `ElasticNet` (`l1_ratio` corresponds to the mix ratio r):

```
>>> from sklearn.linear_model import ElasticNet
>>> elastic_net = ElasticNet(alpha=0.1, l1_ratio=0.5)
>>> elastic_net.fit(X, y)
>>> elastic_net.predict([[1.5]])
array([1.54333232])
```

Which regularization method?

- When should you use plain Linear Regression (i.e., without any regularization), Ridge, Lasso, or Elastic Net?
- It is almost always preferable to have at least a little bit of regularization, so generally you should avoid plain Linear Regression.
- Ridge is a good default, but **if you suspect that only a few features are useful**, **you should prefer Lasso or Elastic Net because they tend to reduce the useless features' weights down to zero**.
- In general, **Elastic Net is preferred over Lasso** because Lasso may behave erratically when the number of features is greater than the number of training instances or when several features are strongly correlated.

Early Stopping

- A very different way to regularize iterative learning algorithms such as Gradient Descent is to **stop training as soon as the validation error reaches a minimum**. This is called early stopping.
- Figure: complex model (in this case, a high-degree Polynomial Regression model) being trained with Batch Gradient Descent.
- It is such a simple and efficient regularization technique that Geoffrey Hinton called it “beautiful free lunch.”

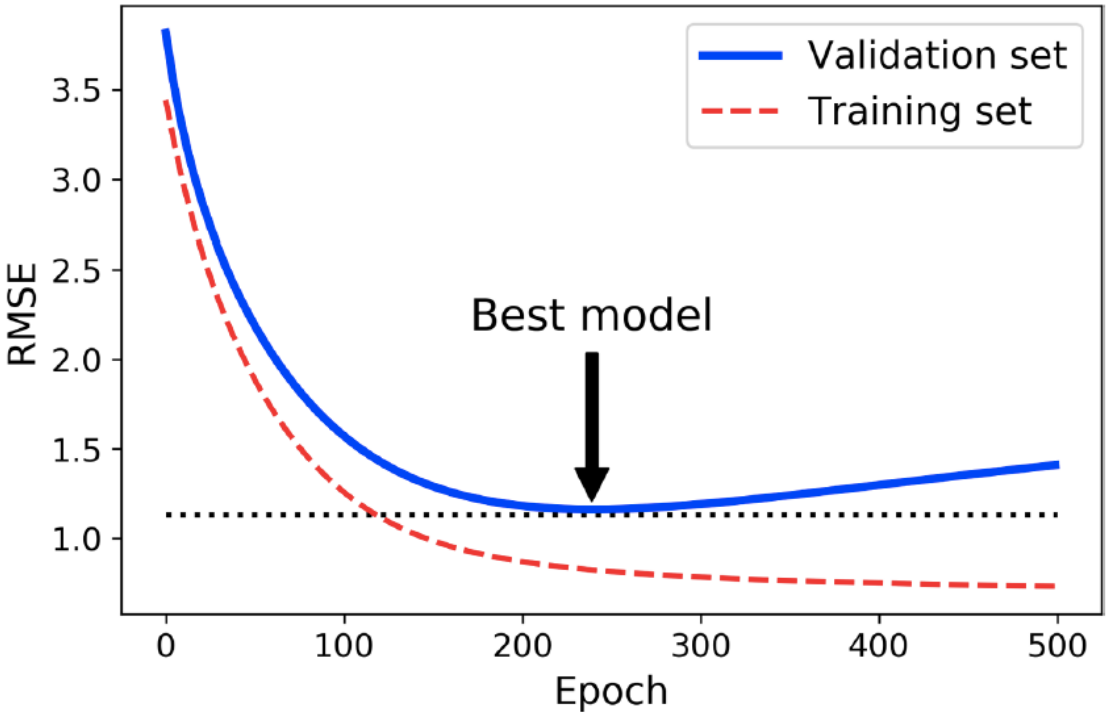


Figure 4-20. Early stopping regularization

Early Stopping - Implementation

- With **stochastic and mini-batch gradient** descent, the curves are not so smooth, and it may be hard to know whether you have reached the minimum or not.
- One solution is to **stop only after the validation error has been above the minimum for some time** (**when you are confident that the model will not do any better**), then roll back the model parameters to the point where the validation error was at a minimum.

Early Stopping

Implementation

```
from copy import deepcopy
from sklearn.metrics import mean_squared_error
from sklearn.preprocessing import StandardScaler

X_train, y_train, X_valid, y_valid = [...]  # split the quadratic dataset

preprocessing = make_pipeline(PolynomialFeatures(degree=90, include_bias=False),
                              StandardScaler())
X_train_prep = preprocessing.fit_transform(X_train)
X_valid_prep = preprocessing.transform(X_valid)
sgd_reg = SGDRegressor(penalty=None, eta0=0.002, random_state=42)
n_epochs = 500
best_valid_rmse = float('inf')

for epoch in range(n_epochs):
    sgd_reg.partial_fit(X_train_prep, y_train)
    y_valid_predict = sgd_reg.predict(X_valid_prep)
    val_error = mean_squared_error(y_valid, y_valid_predict, squared=False)
    if val_error < best_valid_rmse:
        best_valid_rmse = val_error
        best_model = deepcopy(sgd_reg)
```

Early Stopping

Implementation

- This code first adds the polynomial features and scales all the input features, both for the training set and for the validation set (the code assumes that you have split the original training set into a smaller training set and a validation set).
- Then it creates an `SGDRegressor` model with no regularization and a small learning rate.
- In the training loop, it calls `partial_fit()` instead of `fit()`, to perform incremental learning (continues where it left off).
- At each epoch, it measures the RMSE on the validation set. If it is lower than the lowest RMSE seen so far, it saves a copy of the model in the `best_model` variable.
- This implementation **does not actually stop training**, but it lets you revert to the best model after training.
- Note that the model is copied using `copy.deepcopy()`, because it copies both the model's hyperparameters and the learned parameters. In contrast, `sklearn.base.clone()` only copies the model's hyperparameters.

Logistic Regression

Logistic Regression

- Some regression algorithms can be used for classification (and vice versa). Logistic Regression (also called Logit Regression) is commonly used to estimate the probability that an instance belongs to a particular class (e.g., what is the probability that this email is spam?).
- If the estimated probability is greater than 50%, then the model predicts that the instance belongs to that class (called the positive class, labeled “1”), and otherwise it predicts that it does not (i.e., it belongs to the negative class, labeled “0”).
- This makes it a **binary classifier**.

Logistic Regression
Estimating Probabilities

- Logistic Regression model computes a weighted sum of the input features (plus a bias term), but instead of outputting the result directly like the Linear Regression model does, it outputs the logistic of this result using the *sigmoid* function:

Logistic Regression model estimated probability (vectorized form)

$$\hat{p} = h_{\boldsymbol{\theta}}(\mathbf{x}) = \sigma(\mathbf{x}^{\intercal}\boldsymbol{\theta})$$

Equation 4-14. Logistic function

$$\sigma(t) = \frac{1}{1 + \exp(-t)}$$

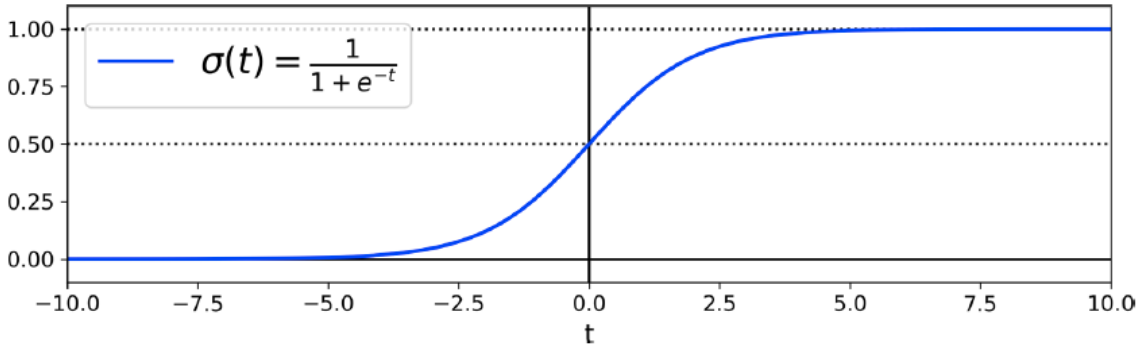


Figure 4-21. Logistic function

Logistic Regression
Estimating Probabilities

Once the Logistic Regression model has estimated the probability $\hat{p} = h_{\boldsymbol{\theta}}(\mathbf{x})$ that an instance $\mathbf{x}$ belongs to the positive class, it can make its prediction $\hat{y}$ easily (see Equation 4-15).

Equation 4-15. Logistic Regression model prediction

$$\hat{y} = \begin{cases} 0 & \text{if } \hat{p} < 0.5 \\ 1 & \text{if } \hat{p} \geq 0.5 \end{cases}$$

Notice that $\sigma(t) < 0.5$ when $t < 0$, and $\sigma(t) \geq 0.5$ when $t \geq 0$, so a Logistic Regression model predicts 1 if $\mathbf{x}^\intercal \boldsymbol{\theta}$ is positive and 0 if it is negative.

NOTE

The score t is often called the *logit*. The name comes from the fact that the logit function, defined as $\text{logit}(p) = \log(p / (1 - p))$, is the inverse of the logistic function. Indeed, if you compute the logit of the estimated probability p, you will find that the result is t. The logit is also called the *log-odds*, since it is the log of the ratio between the estimated probability for the positive class and the estimated probability for the negative class.

Logistic Regression
Training and Cost Function

- The objective of training is to set the parameter vector θ so that the model estimates high probabilities for positive instances (y = 1) and low probabilities for negative instances (y = 0).
- Cost for a single training instance:

$$c(\boldsymbol{\theta}) = \begin{cases} -\log(\hat{p}) & \text{if } y = 1 \\ -\log(1-\hat{p}) & \text{if } y = 0 \end{cases}$$

- Cost function over the whole training set (*log loss*:

$$J(\boldsymbol{\theta}) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}log\left(\hat{p}^{(i)}\right)+\left(1-y^{(i)}\right)log\left(1-\hat{p}^{(i)}\right)\right]$$

Logistic Regression
Training

- The bad news is that there is no known closed-form equation to compute the value of θ that minimizes this cost function.
- The good news is that **this cost function is convex**, so Gradient Descent (or any other optimization algorithm) is guaranteed to find the global minimum (if the learning rate is not too large and you wait long enough).
- The partial derivatives of the cost function with regard to the jth model parameter θ are given by:

$$\frac{\partial}{\partial \theta_j} \mathrm{J}(\boldsymbol{\theta}) = \frac{1}{m} \sum_{i=1}^{m} \left(\sigma\left(\boldsymbol{\theta}^{\mathsf{T}} \mathbf{x}^{(i)}\right) - y^{(i)} \right) x_j^{(i)}$$

For full derivation see:
https://medium.com/analytics-vidhya/derivative-of-log-loss-function-for-logistic-regression-9b832f025c2d

Logistic Regression
Example

- **Iris dataset**: sepal and petal length and width of 150 iris flowers of three species: *Iris setosa*, *Iris versicolor* and *Iris virginica*.

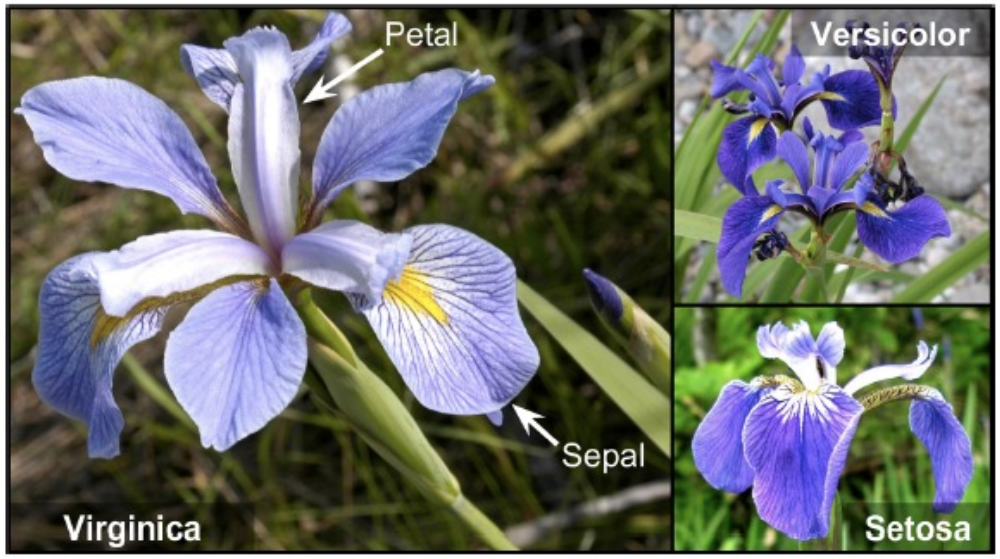


Figure 4-22. Flowers of three iris plant species[14]

Logistic Regression
Example

- **Goal**: Build a classifier to **detect the Iris virginica type based only on the petal width feature**.
- Load the data:

```
>>> from sklearn import datasets
>>> iris = datasets.load_iris()
>>> list(iris.keys())
['data', 'target', 'target_names', 'DESCR', 'feature_names', 'filename']
>>> X = iris["data"][:, 3:]  # petal width
>>> y = (iris["target"] == 2).astype(np.int)  # 1 if Iris virginica, else 0
```

- Train a Logistic regression model:

```
from sklearn.linear_model import LogisticRegression

log_reg = LogisticRegression()
log_reg.fit(X, y)
```

Logistic Regression
Example

- Let's look at the model's estimated probabilities for flowers with petal widths varying from 0 cm to 3 cm:

```
X_new = np.linspace(0, 3, 1000).reshape(-1, 1)
y_proba = log_reg.predict_proba(X_new)
plt.plot(X_new, y_proba[:, 1], "g-", label="Iris virginica")

plt.plot(X_new, y_proba[:, 0], "b--", label="Not Iris virginica")
# + more Matplotlib code to make the image look pretty
```

Logistic Regression
Example

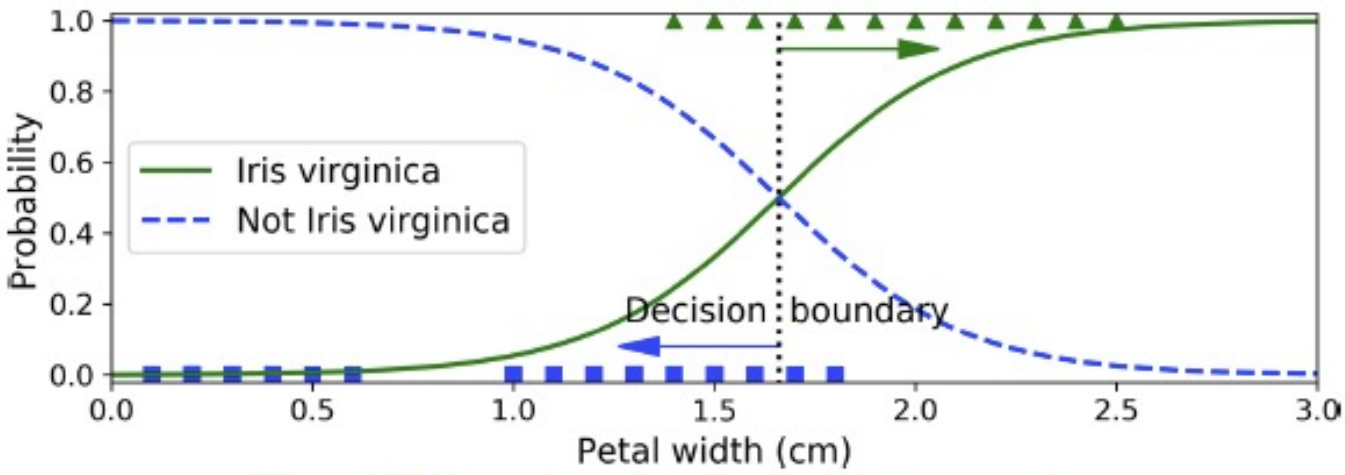


Figure 4-23. Estimated probabilities and decision boundary

- The petal width of Iris virginica flowers (represented by triangles) ranges from 1.4 cm to 2.5 cm, while the other iris flowers (represented by squares) generally have a smaller petal width, ranging from 0.1 cm to 1.8 cm. Notice that there is a bit of overlap.
- Above about 2 cm the classifier is highly confident that the flower is an Iris virginica (it outputs a high probability for that class), while below 1cm it is highly confident that it is not an Iris virginica (high probability for the "Not Iris virginica" class).
- In between these extremes, the classifier is unsure. However, if you ask it to predict the class (using the **predict()** method rather than the **predict_proba()** method), it will return whichever class is the most likely.
- Therefore, there is a decision boundary at around 1.6 cm where both probabilities are equal to 50%: if the petal width is higher than 1.6 cm, the classifier will predict that the flower is an Iris virginica, and otherwise it will predict that it is not (even if it is not very confident):

```
>>> log_reg.predict([[1.7], [1.5]])
array([1, 0])
```

Logistic Regression Example: Iris dataset

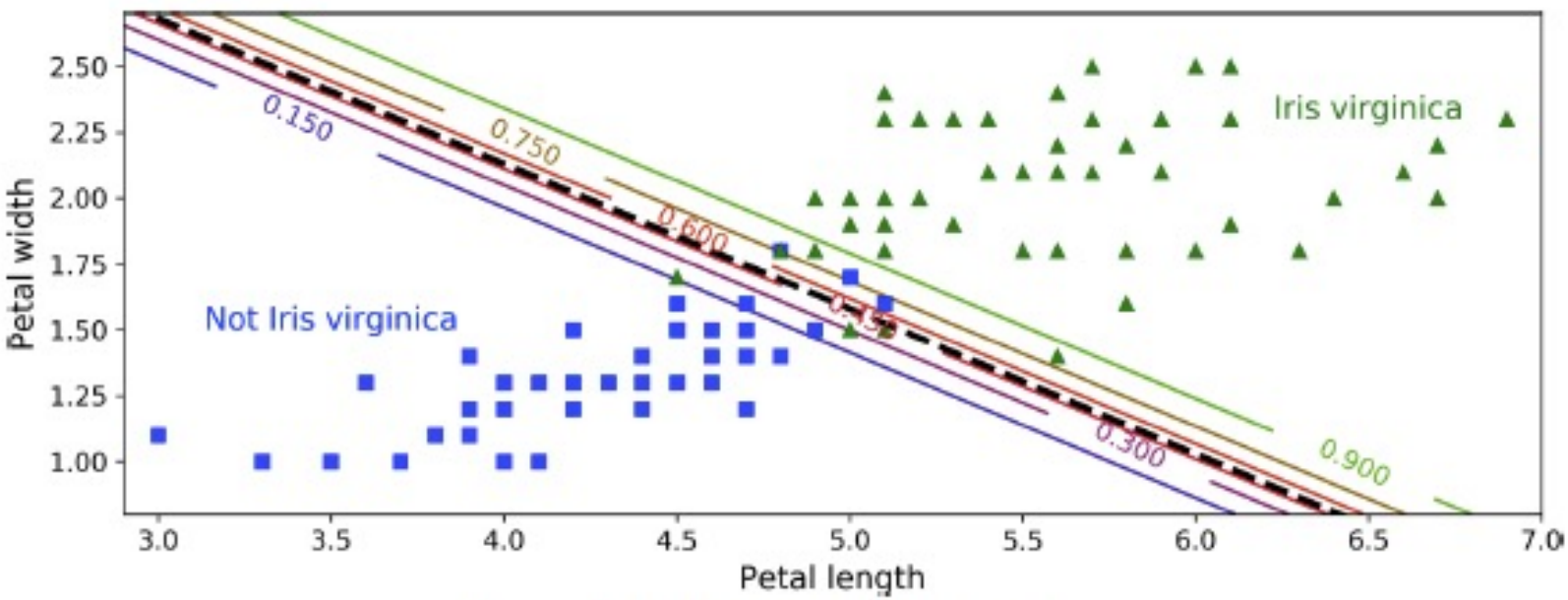


Figure 4-24. Linear decision boundary

- Use **two features: petal width and length** to to classify Iris Virginica vs. not Iris virginica.
- The dashed line represents the points where the model estimates a 50% probability: this is the model's decision boundary. Note that it is a linear boundary.
- Each parallel line represents the points where the model outputs a specific probability, from 15% (bottom left) to 90% (top right). All the flowers beyond the top-right line have an over 90% chance of being Iris virginica, according to the model.

Logistic Regression
Regularization

Just like the other linear models, Logistic Regression models can be regularized using ℓ_1 or ℓ_2 penalties. Scikit-Learn actually adds an ℓ_2 penalty by default.

NOTE

The hyperparameter controlling the regularization strength of a Scikit-Learn `LogisticRegression` model is not `alpha` (as in other linear models), but its inverse: `C`. The higher the value of `C`, the *less* the model is regularized.

Softmax Regression
(Multinomial Logistic Regression)

- The Logistic Regression model can be **generalized to support multiple classes directly**, without having to train and combine multiple binary classifiers.
- The idea is simple: when given an instance **x**, the Softmax Regression model first computes a score $s_k(\mathbf{x})$ for each class k, then estimates the probability of each class by applying the softmax function (also called the normalized exponential) to the scores.
- The equation to compute $s_k(\mathbf{x})$ should look familiar, as it is just like the equation for Linear Regression prediction:

Softmax score for class k

$$s_k(\mathbf{x}) = \mathbf{x}^T \boldsymbol{\theta}^{(k)}$$

Softmax Regression (Multinomial Logistic Regression)

- Once you have computed the score of every class for the instance x, you can estimate the probability $\hat{p}_k$ that the instance belongs to class k by running the scores through the softmax function (Equation 4-20).
- The function computes the exponential of every score, then normalizes them (dividing by the sum of all the exponentials). The scores are generally called *logits* or *log-odds*.

Equation 4-20. Softmax function

$$\hat{p}_k = \sigma(\mathbf{s}(\mathbf{x}))_k = \frac{\exp(s_k(\mathbf{x}))}{\sum_{j=1}^{K} \exp(s_j(\mathbf{x}))}$$

In this equation:

- K is the number of classes.
- $\mathbf{s}(\mathbf{x})$ is a vector containing the scores of each class for the instance $\mathbf{x}$.
- $\sigma(\mathbf{s}(\mathbf{x}))_k$ is the estimated probability that the instance $\mathbf{x}$ belongs to class k, given the scores of each class for that instance.

Softmax Regression
(Multinomial Logistic Regression)

- Just like the Logistic Regression classifier, the Softmax Regression classifier predicts the class with the highest estimated probability (which is simply the class with the highest score):

$$\hat{y} = \underset{k}{\operatorname{argmax}}\ \sigma(\mathbf{s}(\mathbf{x}))_k = \underset{k}{\operatorname{argmax}}\ s_k(\mathbf{x}) = \underset{k}{\operatorname{argmax}}\left(\left(\boldsymbol{\theta}^{(k)}\right)^{\mathsf{T}}\mathbf{x}\right)$$

The *argmax* operator returns the value of a variable that maximizes a function. In this equation, it returns the value of k that maximizes the estimated probability $\sigma(\mathbf{s}(\mathbf{x}))_k$.

Softmax Regression
Training

- Minimizing the cost function shown in Equation 4-22, called the cross entropy, should lead to this objective because it penalizes the model when it estimates a low probability for a target class.
- Cross entropy is frequently used to measure how well a set of estimated class probabilities matches the target classes.

Equation 4-22. Cross entropy cost function

$$J(\mathbf{\Theta}) = -\frac{1}{m}\sum_{i=1}^{m}\sum_{k=1}^{K} y_k^{(i)} \log\left(\hat{p}_k^{(i)}\right)$$

In this equation:

- $y_k^{(i)}$ is the target probability that the ith instance belongs to class k. In general, it is either equal to 1 or 0, depending on whether the instance belongs to the class or not.

Notice that when there are just two classes ($K = 2$), this cost function is equivalent to the Logistic Regression's cost function (log loss; see Equation 4-17).

CROSS ENTROPY

Cross entropy originated from information theory. Suppose you want to efficiently transmit information about the weather every day. If there are eight options (sunny, rainy, etc.), you could encode each option using three bits because $2^3 = 8$. However, if you think it will be sunny almost every day, it would be much more efficient to code "sunny" on just one bit (0) and the other seven options on four bits (starting with a 1). Cross entropy measures the average number of bits you actually send per option. If your assumption about the weather is perfect, cross entropy will be equal to the entropy of the weather itself (i.e., its intrinsic unpredictability). But if your assumptions are wrong (e.g., if it rains often), cross entropy will be greater by an amount called the *Kullback–Leibler (KL) divergence*.

The cross entropy between two probability distributions p and q is defined as $H(p, q) = -\sum_x p(x) \log q(x)$ (at least when the distributions are discrete). For more details, check out my video on the subject.

https://www.youtube.com/watch?v=ErfnhcEV1O8

Softmax Regression

Training - Optimization

Equation 4-22. Cross entropy cost function

$$J(\mathbf{\Theta}) = -\frac{1}{m}\sum_{i=1}^{m}\sum_{k=1}^{K} y_k^{(i)} \log\left(\hat{p}_k^{(i)}\right)$$

Equation 4-23. Cross entropy gradient vector for class k

$$\nabla_{\boldsymbol{\theta}^{(k)}} J(\mathbf{\Theta}) = \frac{1}{m}\sum_{i=1}^{m}\left(\hat{p}_k^{(i)} - y_k^{(i)}\right)\mathbf{x}^{(i)}$$

Now you can compute the gradient vector for every class, then use Gradient Descent (or any other optimization algorithm) to find the parameter matrix $\mathbf{\Theta}$ that minimizes the cost function.

Softmax Regression

Example: Iris Dataset

Let's use Softmax Regression to classify the iris flowers into all three classes. Scikit-Learn's `LogisticRegression` uses one-versus-the-rest by default when you train it on more than two classes, but you can set the `multi_class` hyperparameter to `"multinomial"` to switch it to Softmax Regression. You must also specify a solver that supports Softmax Regression, such as the `"lbfgs"` solver (see Scikit-Learn's documentation for more details). It also applies ℓ_2 regularization by default, which you can control using the hyperparameter `C`:

```
X = iris["data"][:, (2, 3)]  # petal length, petal width
y = iris["target"]

softmax_reg = LogisticRegression(multi_class="multinomial",solver="lbfgs",
C=10)
softmax_reg.fit(X, y)
```

Softmax Regression

Example: Iris Dataset

So the next time you find an iris with petals that are 5 cm long and 2 cm wide, you can ask your model to tell you what type of iris it is, and it will answer *Iris virginica* (class 2) with 94.2% probability (or *Iris versicolor* with 5.8% probability):

```
>>> softmax_reg.predict([[5, 2]])
array([2])
>>> softmax_reg.predict_proba([[5, 2]])
array([[6.38014896e-07, 5.74929995e-02, 9.42506362e-01]])
```

Softmax Regression Example: Iris Dataset

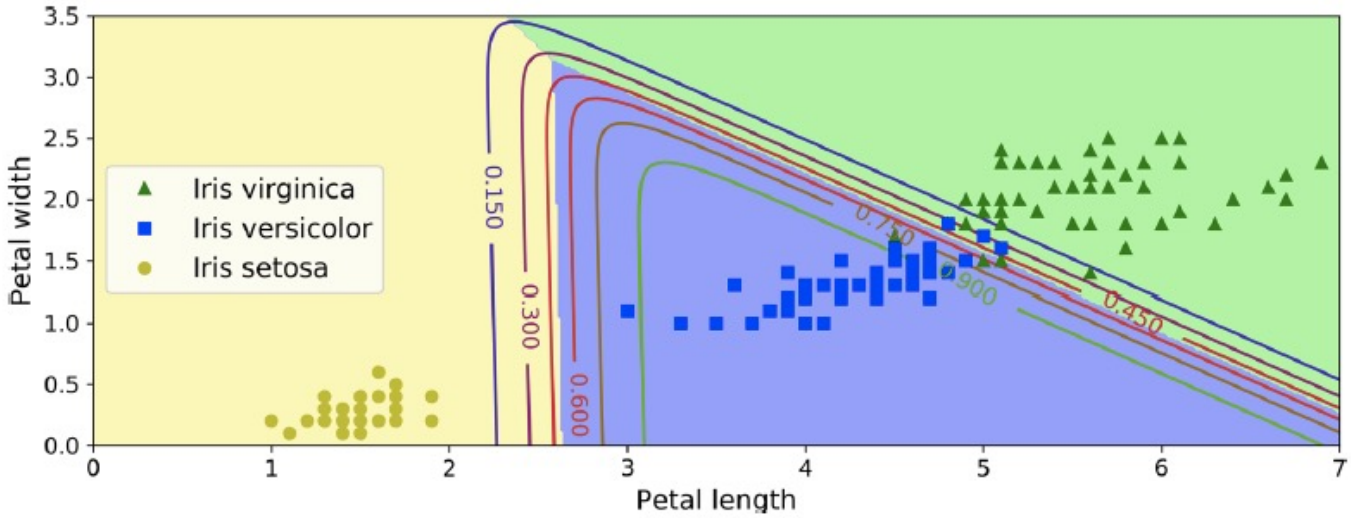


Figure 4-25. Softmax Regression decision boundaries

- The decision boundaries between any two classes are linear.
- The probabilities for the Iris versicolor class, represented by the curved lines (e.g., the line labeled with 0.450 represents the 45% probability boundary).
- Notice that the model can predict a class that has an estimated probability below 50%. For example, at the point where all decision boundaries meet, all classes have an equal estimated probability of 33%.